

现代交换原理

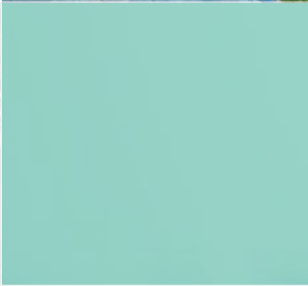
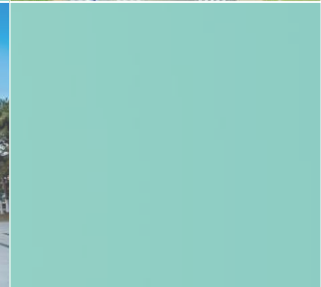
北京邮电大学

计算机学院（国家示范性软件学院）



第六章

软件定义网络

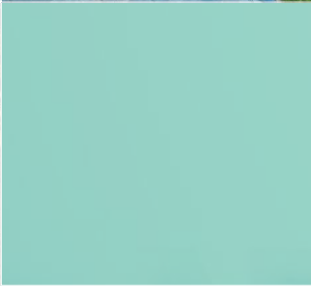
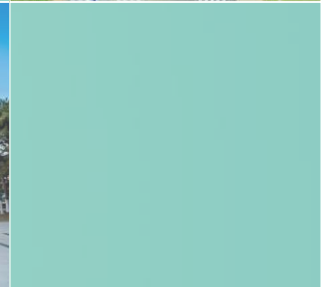


本章内容

- SDN概述
- SDN体系架构
- SDN工作原理
- SDN的应用

SDN概述

01



网络体系寻求变革



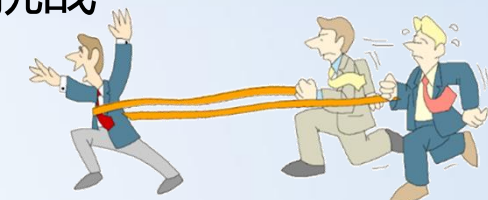
北京邮电大学
BEIHANG UNIVERSITY OF POSTS AND TELECOMMUNICATIONS

服务器



开放的架构：
软硬件分离，
开发新业务
灵活！

新业务层出不穷，对传统网络提出新挑战



信息技术IT 通信技术CT

业务和网络天然割裂

用户和业务的多样性需求

universal communication?

small devices?

authentication?

dependability?

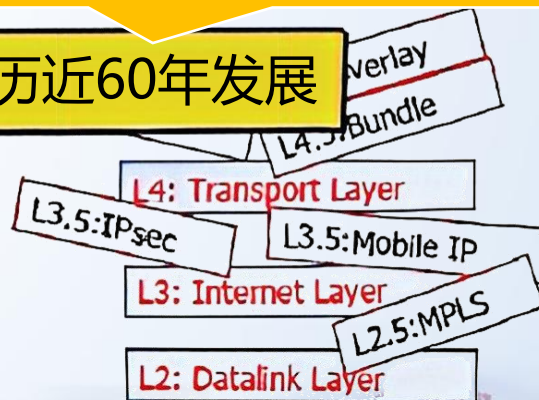
guaranteed service?

传统网络架构封闭保守，网络
创新缓慢、配置繁琐、效率低下

Internet已经历近60年发展



Original Internet Architecture



路由器/
交换机



封闭的设备：
软硬件绑定，
开发新功能
困难！

封闭的通信设备：开发新功能困难



路由器/
交换机

特性 1

特性 2

设备配置与管理

- 静态手动配置
- 特性迭代速度慢

专有
网络操作系统

操作系统

- API接口极少，仅支持命令行界面（封闭式操作系统）
- 不具备外部可编程性

专有硬件系统

硬件系统

- 锁定特定供应商

专有芯片

芯片

- 创新周期缓慢
- 成本高昂，缺乏规模经济

开放的计算架构：开发新业务灵活



服务器

- 网络启动
- 集中配置
- 自动补丁管理

配置与管理

- 网络启动
- 集中式配置与管理

Linux/Windows

操作系统

- 开源或闭源
- 虚拟化或裸机
- 支持多种模型

VMware KVM Xen

Dell

HP

浪潮

硬件系统

- 激烈的竞争
- 品牌产品或“白盒”产品

Intel

AMD

芯片

- 竞争与快速创新

新业务对传统网络提出新挑战

互联网业务

基础网络

产品调研

海量互联网用户，**及时反馈**

依赖物理科学发展，**十年如一日**

产品设计

微创新**每年上线上百款**

商用产品**每2-3年出一款**

产品开发

快速出模，**敏捷迭代**

运营成熟，**商业化>2年**

产品发布

灰度发布，**及时回滚**

开局失败，**前功尽弃**

产品消退

长尾持续，**原地复活**

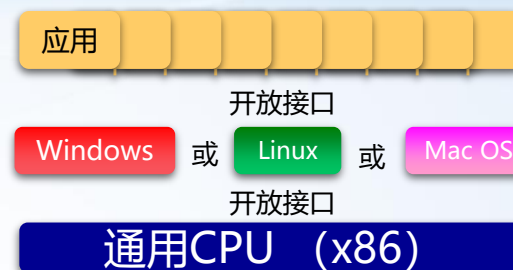
持续运营，**存活>5年**

顺势而为：从封闭走向开放（垂直解耦）

个人电脑



封闭、私有、创新慢



开放、标准、创新快

手机

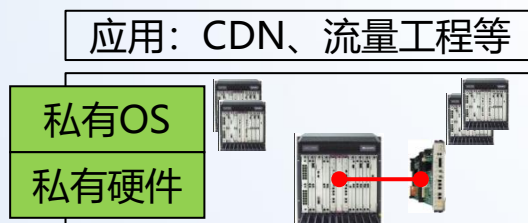


封闭、私有、创新慢、用户体验差

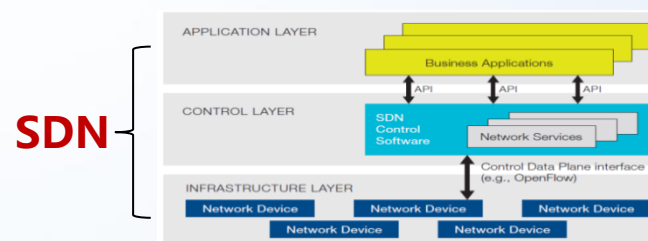


开放/半开放、接口标准、创新快、用户体验好

通信产品



封闭、网络与应用隔离、设备软硬件耦合，创新成本高



开放，网络可被应用调用、承载控制分离、快速创新

未来网络务必要实现开放，满足业务系统的快速定制、自动化和高效率诉求：Software Defined Networking

SDN定义

一组直接使能**编程、协同、控制和管理网络资源的技术**，以利于以动态和可扩展的方式对网络资源设计、交付和运营。-----ITU-T Y.3300 (Y.SDN-FR): Feb. 2014

一种设计、建设和运营用于支持**服务智能性网络**的新方法，**SDN** 将类似于已经用在服务器设施上的抽象、虚拟化和协同的智能性带到网络。-----Gartner

- 软件定义网络（Software Defined Networking, SDN）是一种新型的网络技术，其设计理念是**将网络的控制平面与数据转发平面进行分离**，并实现可编程化的集中控制
- 传统网络设备紧耦合的网络架构被分拆成**应用、控制、转发三层分离的架构**；控制功能被转移到了服务器，上层应用、底层转发设施被抽象成多个逻辑实体

SDN不是一种具体的技术，而是一种思想，一种理念

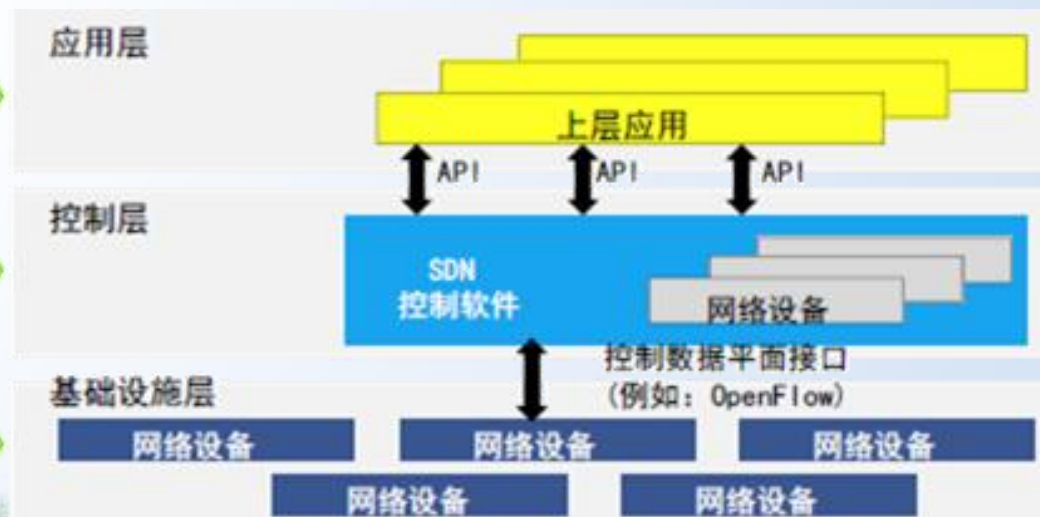
SDN的核心诉求：让软件应用参与到网络控制中并起到主导作用，而不是让而各种固定模式的协议来控制网络

为了满足这种核心诉求，SDN思想指导下的网络必须设计一种新的架构

应用层，不同的应用逻辑通过控制层开放的API管理能力控制设备的报文转发功能

控制层，由SDN控制软件组成，与下层可用OpenFlow协议通信

基础设施层，由转发设备组成



SDN起源与早期技术探索 (2006-2011)



北京邮电大学
BEIHANG UNIVERSITY OF POSTS AND TELECOMMUNICATIONS

01

2006年斯坦福大学启动Clean Slate项目，目标重塑互联网架构，Martin Casado参与并领导**Ethane项目**，通过集中式控制器实现企业网络安全策略统一管理，成为SDN雏形。

02

2007年Martin Casado发表“Ethane: Taking Control of the Enterprise”论文引发学术关注，同年联合创建Nicira公司推动SDN技术向工业界延伸。

03

2008年Nick McKeown发表“**OpenFlow**: Enabling Innovation in Campus Networks”，提出OpenFlow协议实现控制与转发分离，为SDN提供关键技术支撑。

04

2011年Google等7家企业成立**开放网络基金会 (ONF)**，正式推动SDN架构规范与技术发展，奠定行业标准化基础。

SDN技术突破与全球商用落地 (2012-2015)

01

2012年Google B4网络全面运行OpenFlow，**广域网利用率从30%提升至近饱和**，验证SDN商业可行性；VMware以12.6亿美元收购Nicira，加速技术市场化。

02

2013年Cisco、IBM等发起OpenDaylight开源项目，开发SDN控制器及接口，打破设备商技术垄断；Google发布“B4: Experience with a Globally-Deployed Software Defined WAN”，公开SDN数据中心流量优化方案。

03

2014年中国三大运营商开启SDN商用：中国电信与华为完成全球首个运营商SDN部署；中国移动改造政企专线PTN网络，中国联通实现SDN IPRAN商用，提升运维效率与带宽利用率。

04

2015年中国联通发布CUBE-Net 2.0白皮书，提出基于SDN/NFV、云和超宽带的网络重构目标，推动传统网络向智能化转型。

SDN演进创新与前沿技术融合 (2017-2023)

01

2017年ONF启动CORD项目推动电信边缘网络云化，3GPP将SDN/NFV纳入5G核心网，支持网络切片与动态资源管理，**开启SDN与5G深度融合。**

02

2019年P4语言v1.0规范发布，成为可编程交换机与数据平面行业标准；2020年ONF推出开源5G边缘云平台Aether，基于SDN与云原生技术支持企业专网与边缘计算。

03

2021年开放可编程网络（OPN）框架提出，结合P4、eBPF等技术实现全栈可编程；2023年IETF发布SRv6增强标准（RFC 9481），完善5G核心网与云骨干网的流量工程能力。

04

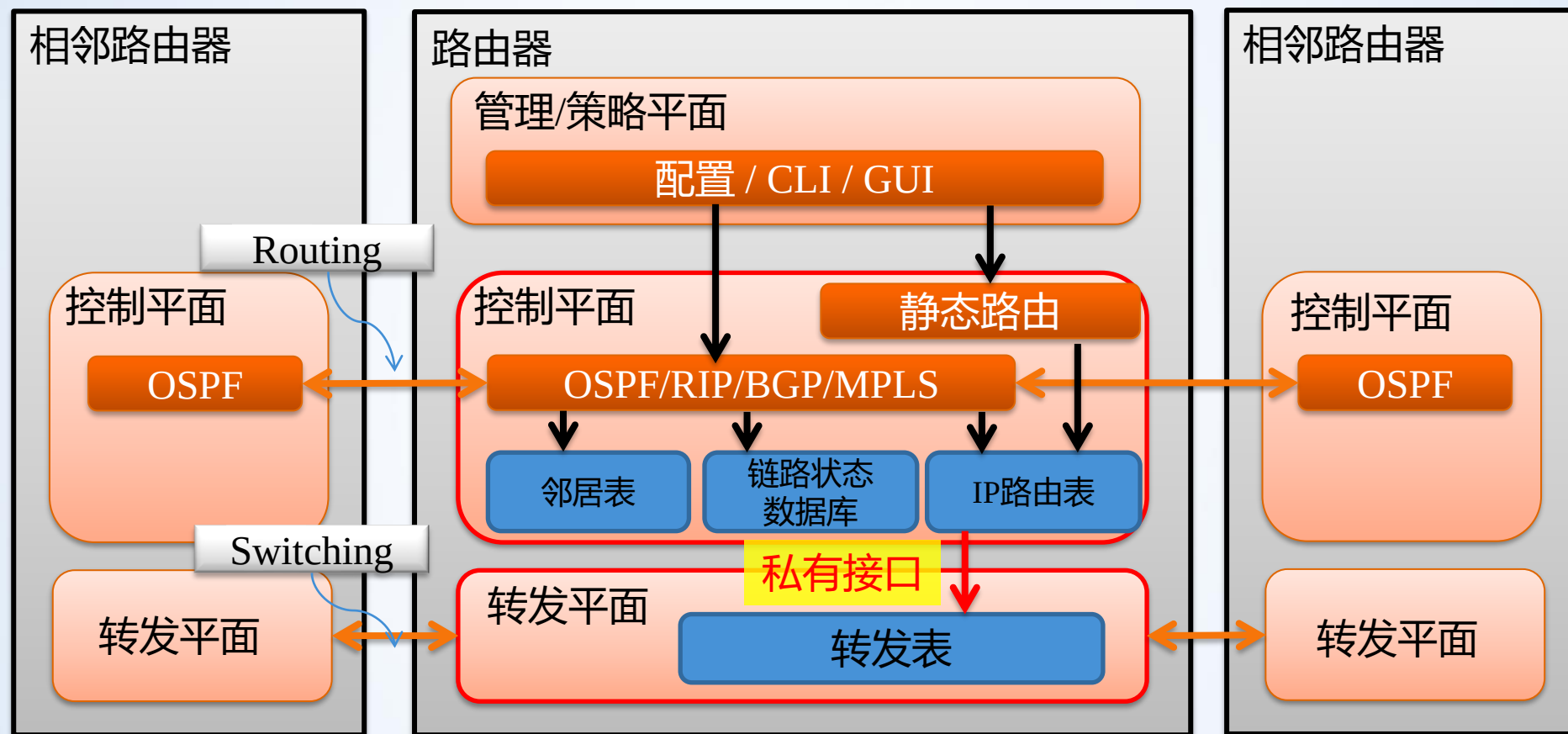
2023年华为推出AI原生SDN控制器“iMaster NCE 3.0”，通过大模型实现自然语言驱动网络策略生成，推动SDN向智能化、自动化升级。

SDN体系架构

02



传统网络节点: Router/Switch



CLI: 命令行界面
GUI: 图形用户界面

软件定义网络 (SDN) 三层架构



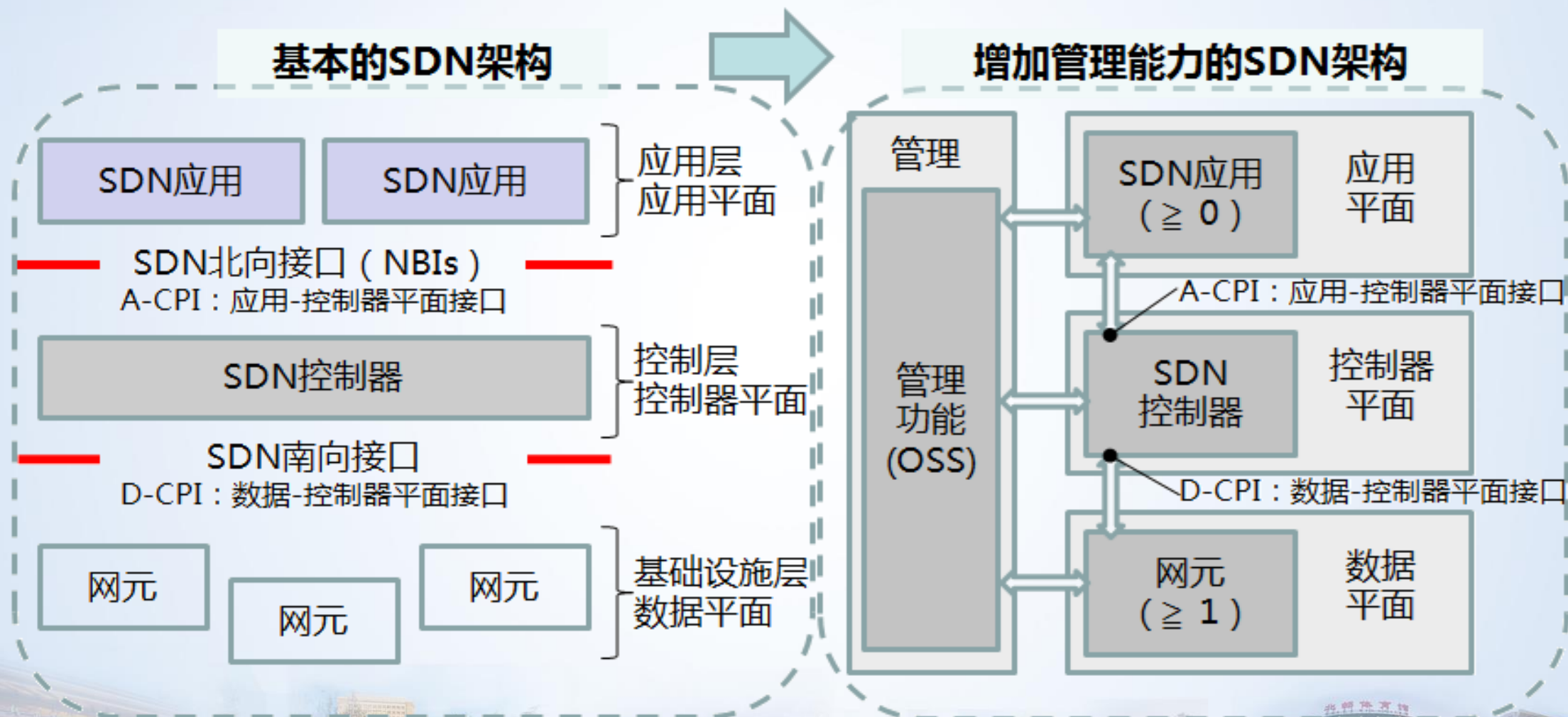
业务平面

控制平面

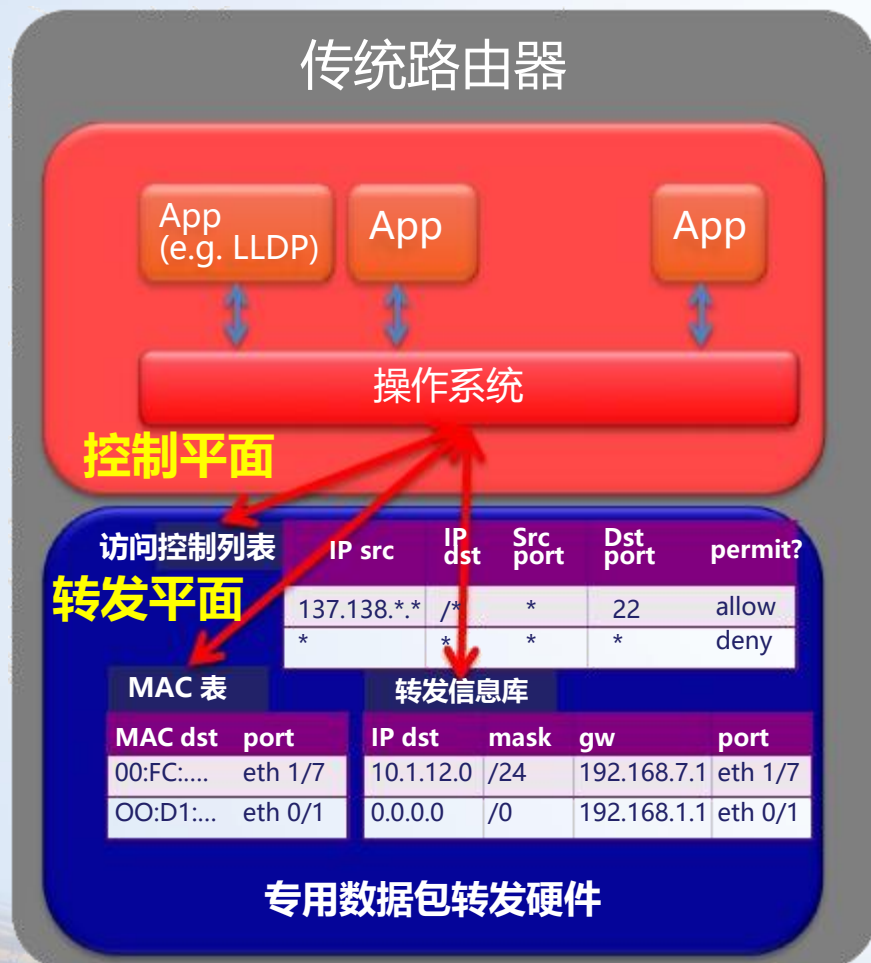
转发平面

软件定义网络 (SDN) 三层架构

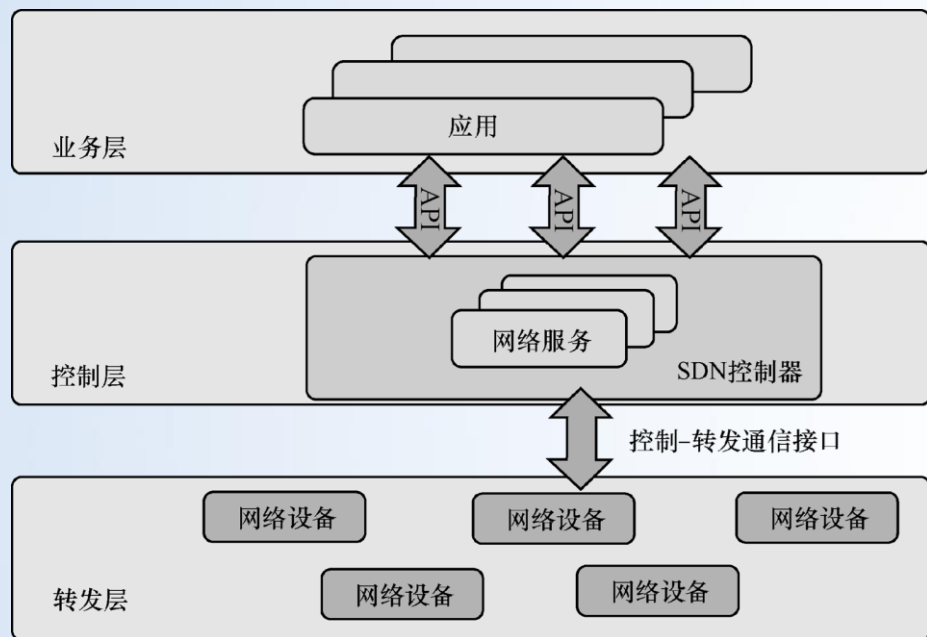
- SDN = **控制与转发分离** + **逻辑集中控制** + **网络资源能力状态开放**



传统路由器与SDN内部功能对比



ONF与IETF: SDN标准化的架构解析



SDN的基本架构

ONF (开放网络基金会) 由互联网巨头联合发起
IETF (互联网工程任务组) 是互联网标准的传统捍卫者

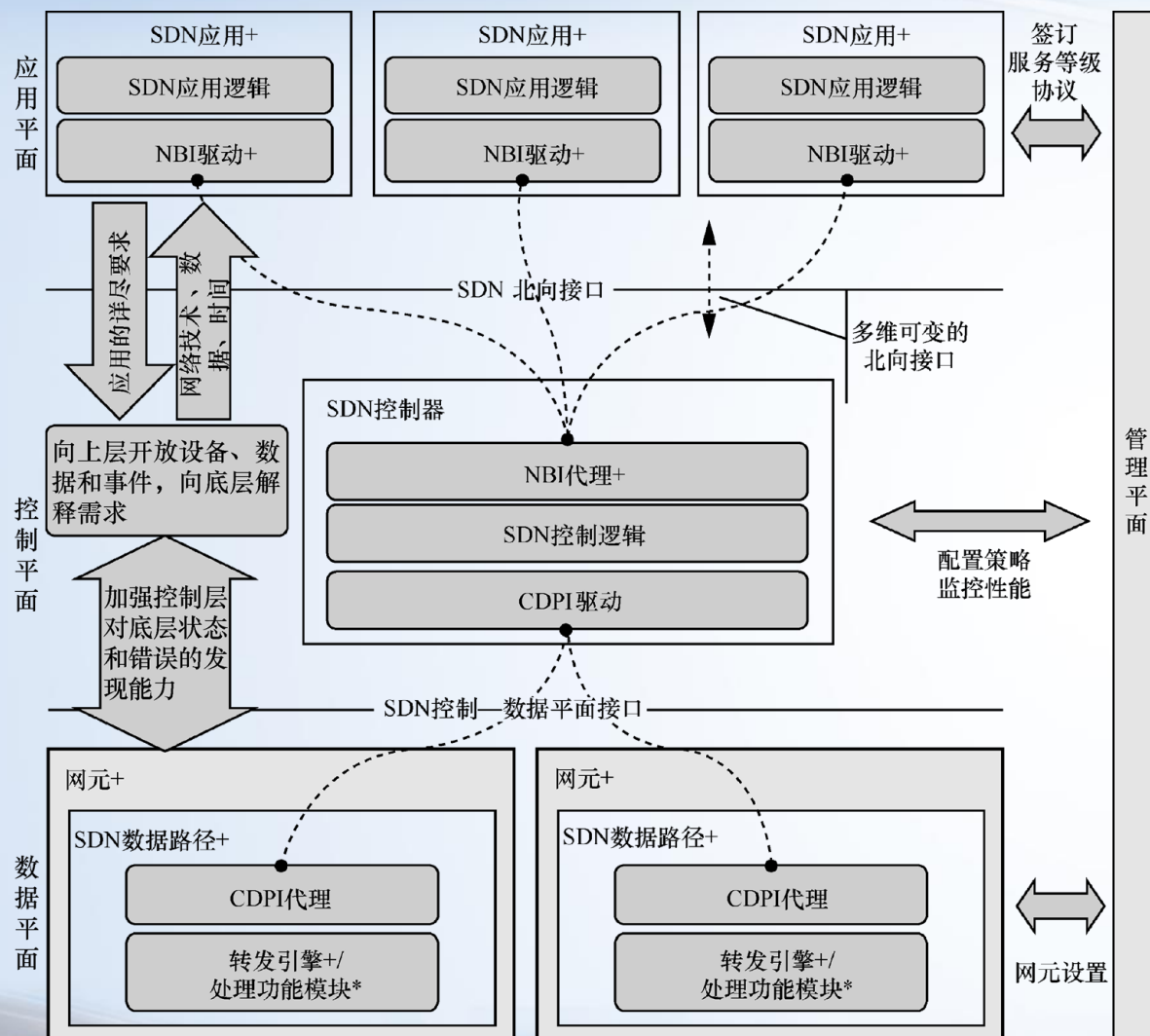
组织背景与共通愿景

- ONF和IETF是并立的SDN标准化力量, ONF旨在通过“白盒化”彻底推翻传统硬件厂商的束缚, IETF倾向在现有协议基础上进行演进
- 两者的架构在核心逻辑上高度一致: 致力于实现控制平面与转发平面的解耦, 从而通过集中的逻辑控制和北向接口的抽象, 实现网络流量的灵活调度与软件化编程

架构差异与演进哲学

- ONF的架构更具革命性, 它深度绑定OpenFlow协议, 追求极致的中心化控制, 试图将底层设备简化为只听指令的“哑终端”
- IETF的RFC 7426架构则更加务实且模块化, 显式地定义了管理平面 (Management Plane) 的独立性, 并允许控制平面以分布式或混合模式存在

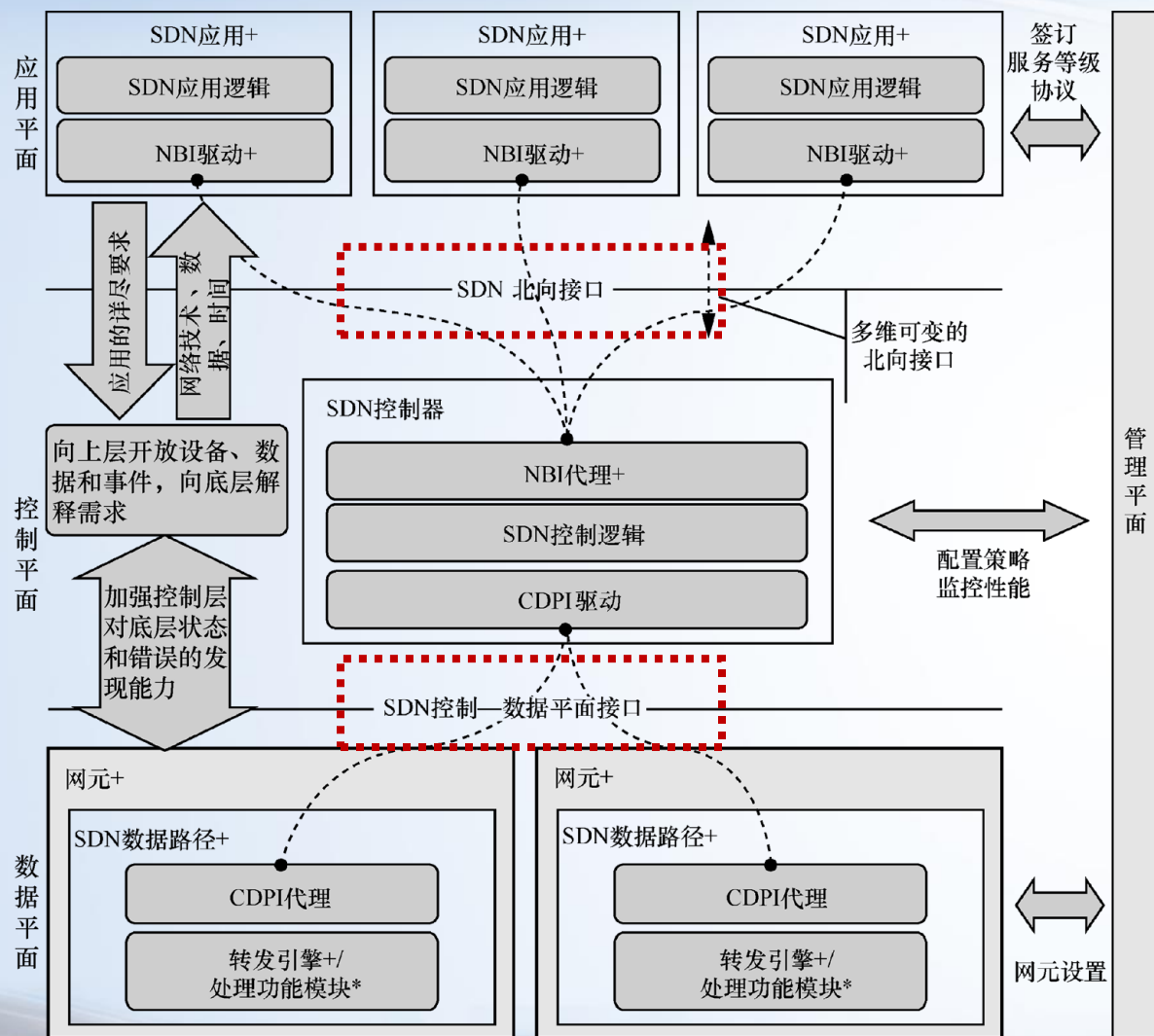
ONF定义的SDN架构



注: + 表示此处可以有一个或多个该实例 | * 表示此处可以有零个或多个该实例。

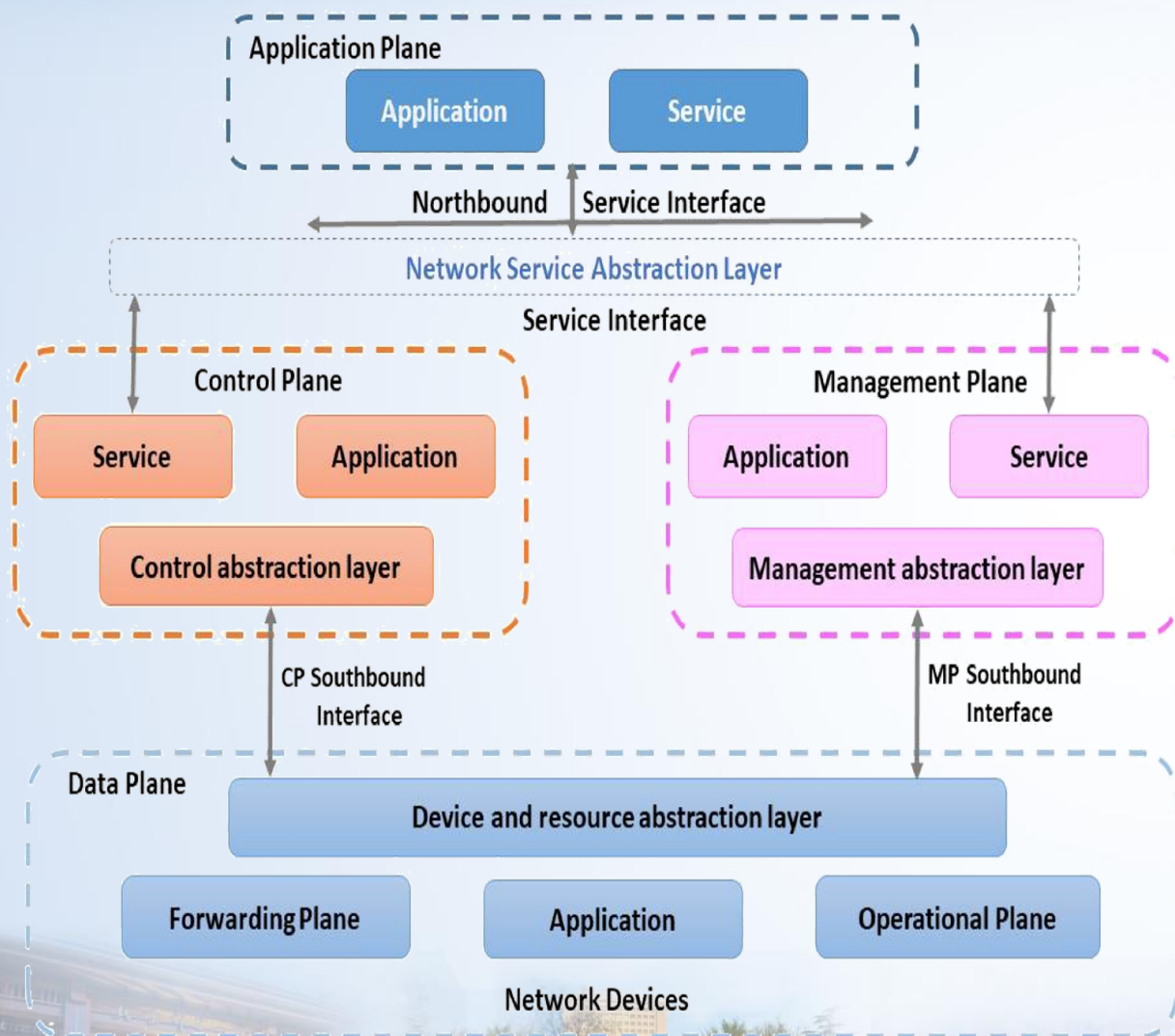
- **数据平面:** 数据平面由若干网元 (Net Element) 构成, 每个网元可以包含一个或多个SDN数据路径SDN Datapath), 是一个被管理的资源在逻辑上的抽象集合
- **控制平面:** 控制平面为SDN控制器, SDN控制器是一个逻辑上集中的实体, 它主要承担两个任务: 一是将SDN应用层请求转换到SDN数据路径, 二是为SDN应用提供底层网络的抽象模型
- **应用平面:** 应用平面由若干用户需要的SDN应用 (SDN Application) 构成, 它可以通过北向接口与SDN控制器进行交互, 即这些应用能够通过可编程方式把需要请求的网络行为提交给控制器
- **管理平面:** 管理平面主要负责一系列静态的工作, 这些工作比较适合在应用、控制、数据平面外实现

ONF定义的SDN架构



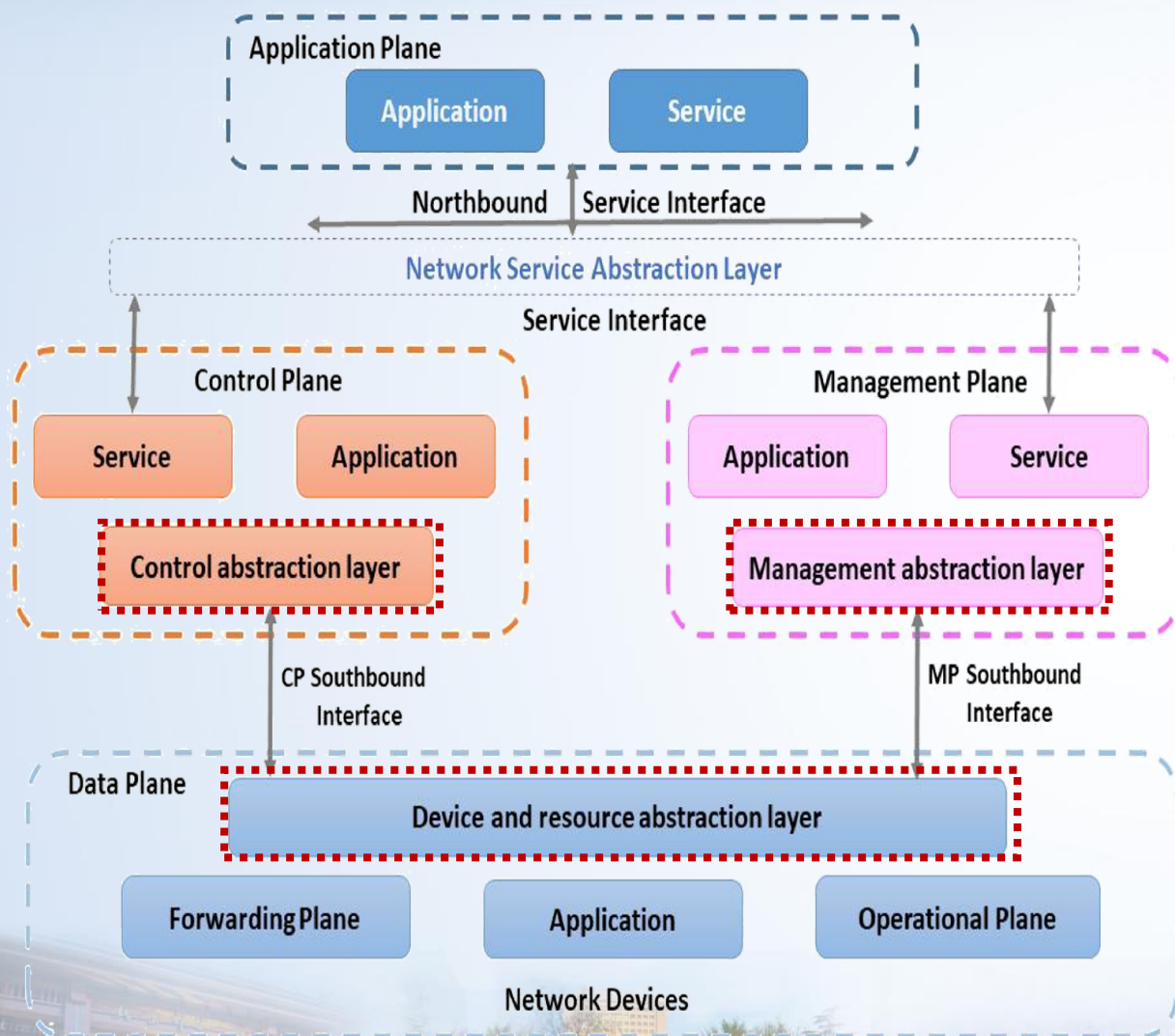
- **SDN控制-数据平面接口:** 是控制平面和数据平面之间的接口。它提供的主要功能包括对所有转发行为进行控制、设备性能查询、统计报告、事件通知等
- **SDN北向接口:** SDN北向接口是应用平面和控制平面之间的一系列接口。它主要负责提供抽象的网络视图, 并使应用能直接控制网络的行为, 其中包含从不同层次对网络及功能进行的抽象。这个接口也应该是一个开放的、与厂商无关的接口

IETF定义的SDN架构 (RFC 7426)



- **数据平面（网络设备平面）：** 由所有执行数据包处理和转发的物理或虚拟网络设备构成，每个设备都包含两个关键子平面，负责高速转发数据的转发平面，以及负责设备本地运行状态（如端口、队列、电源）的操作平面
- **控制平面：** 被定义为一组控制网络设备的功能集合，其核心职责是接收应用平面的指令，通过计算、决策，转化为具体的网络规则（如流表），并下发给数据平面设备
- **应用平面：** 由一系列应用和服务构成。应用是执行特定网络功能（如负载均衡、安全策略）的独立软件；服务则是提供可复用功能模块、并对外提供API的软件单元。应用平面通过开放的北向接口，将高层的业务目标“翻译”并下达给控制平面
- **管理平面：** 面向网络运维，负责对网络设备或设备部件进行监控、配置和维护。与管理平面交互的主要是设备内部的操作平面，而非转发平面

IETF定义的SDN架构 (RFC 7426)

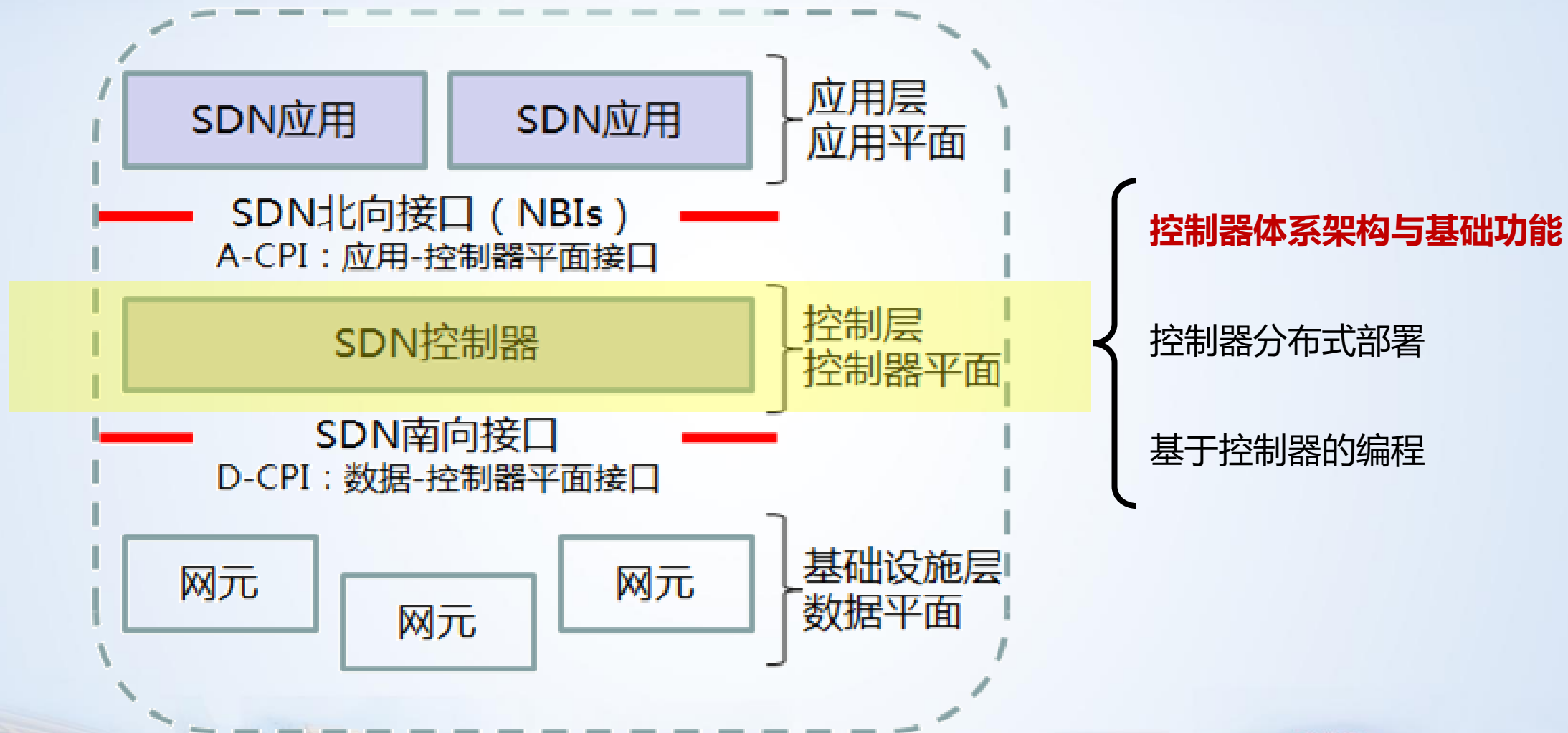


- **设备与资源抽象层**：位于网络设备之上，为控制和管理平面提供统一的、标准化的设备资源视图，隐藏底层硬件的具体差异
- **控制抽象层与管理抽象层**：分别服务于控制平面和管理平面，作为它们与南向接口交互的中间层，进一步封装底层协议的细节
- **南向接口**：控制平面与数据平面之间通过**控制-数据平面接口**交互（如OpenFlow），用于实时、动态的流控制；而管理平面与数据平面之间则是**管理-数据平面接口**（如NETCONF/YANG），用于相对静态的设备配置与状态查询
- **北向接口**：是一组开放的、可编程的API，其核心功能是将网络能力抽象为服务，供上层应用调用，从而实现业务意图对网络的动态、灵活驱动

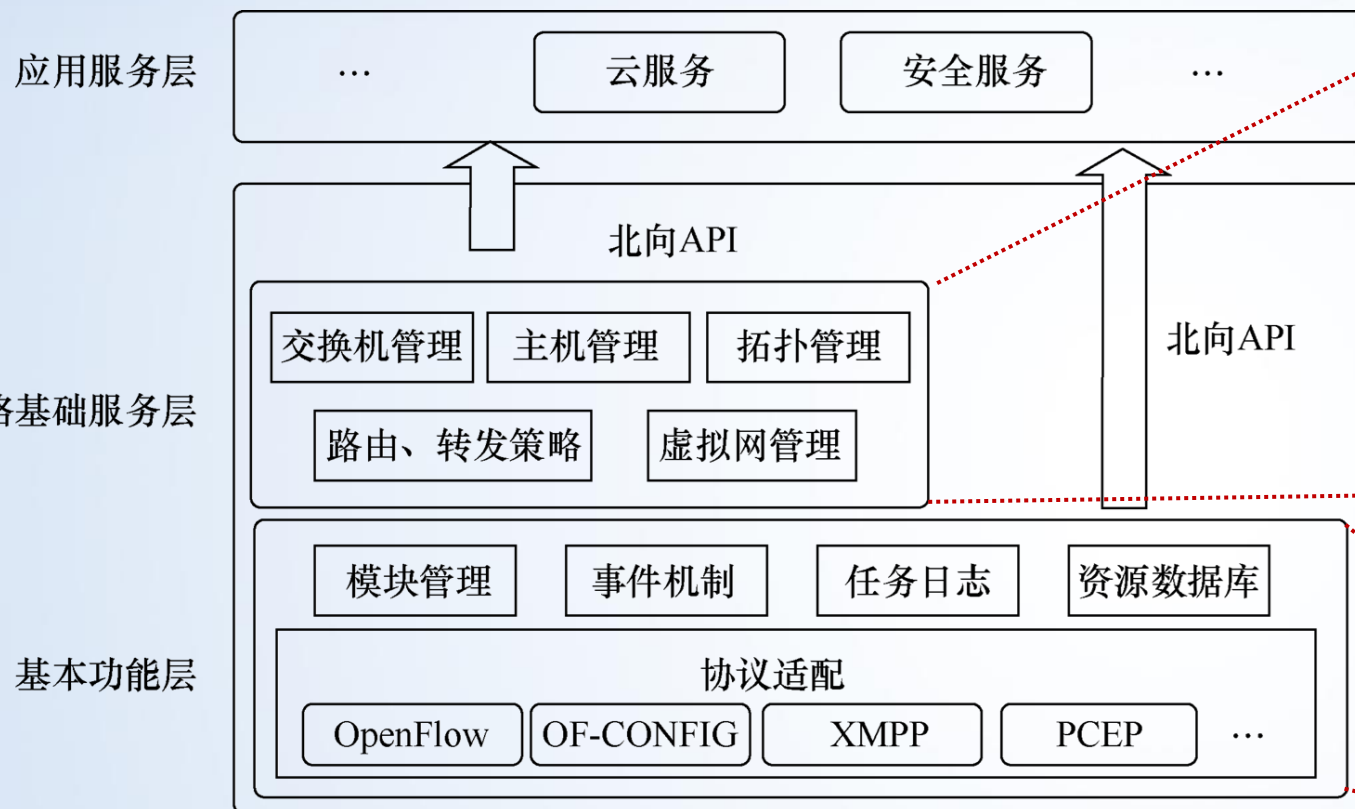
SDN工作原理

03





SDN控制器层次化架构



网络基础服务层：实现控制器的网络智能

- **交换机管理与主机管理**，分别对网络中的转发设备和终端进行发现与状态维护
- **拓扑管理**，自动发现并构建全网连接视图
- **路由和转发策略**，基于全网视图进行路径计算并下发流表
- **虚拟网管理**，实现网络资源的逻辑切片与隔离

基本功能层：核心是协议适配模块，支持OpenFlow、OF-CONFIG、PCEP、XMPP等多种南向协议

SDN控制器的共性架构

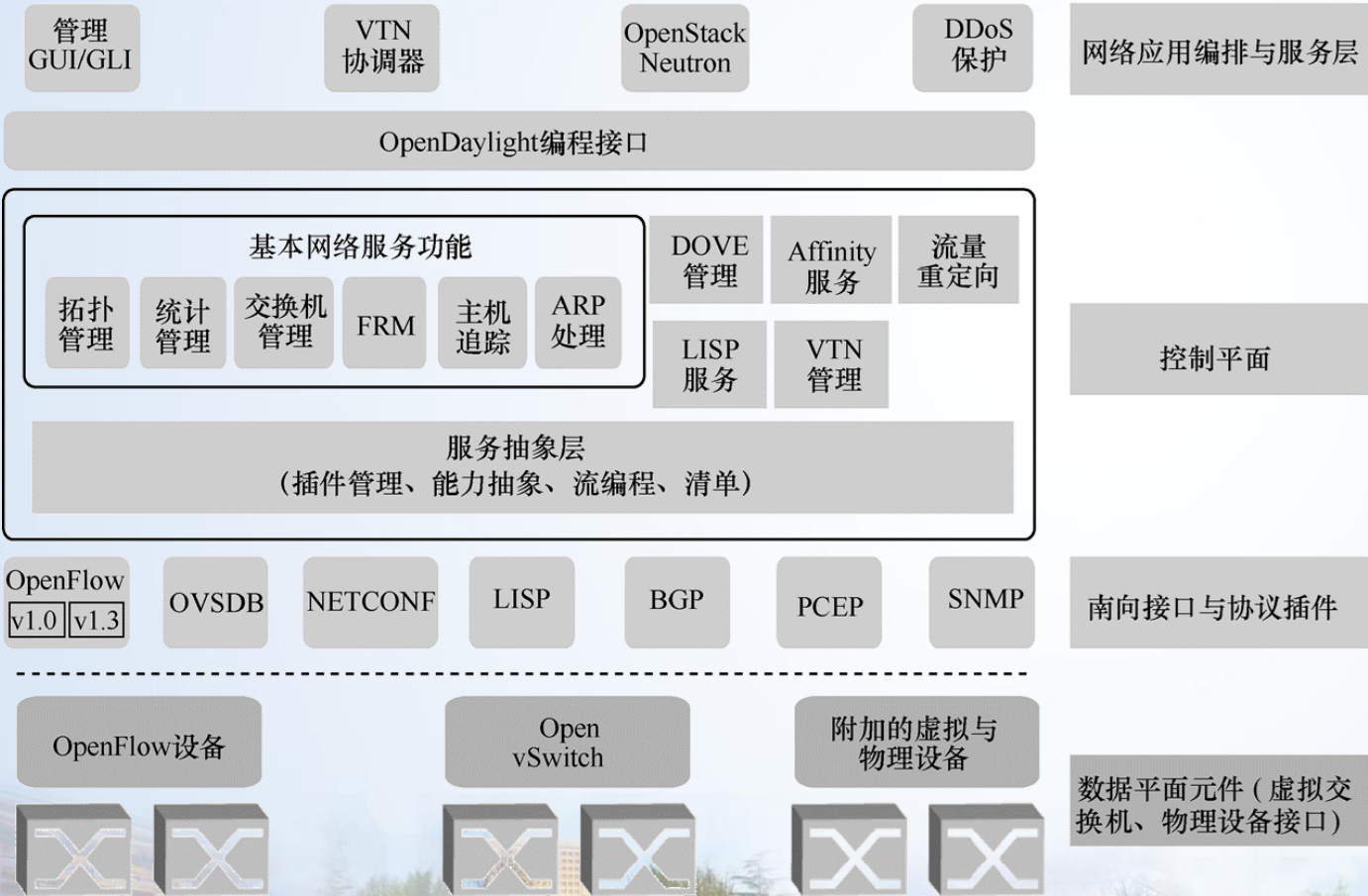
(OpenDaylight、Ryu等控制器具体架构各异)

- **OpenDaylight (ODL)** : Linux基金会, 基于Java的模块化控制器, 支持多协议 (如 OpenFlow、NETCONF), 提供灵活的可扩展框架
- **ONOS**: ONF (开放网络基金会) 与Linux基金会, 面向运营商级网络的分布式控制器, 强调高可用性、性能和横向扩展能力
- **RYU**: 日本NTT社区开发, 轻量级Python控制器, 专注于OpenFlow支持, 提供简洁的API便于快速开发
- **Floodlight**: Big Switch Networks开源社区, 基于Java的OpenFlow控制器, 适用于企业级及数据中心场景, 提供REST API与模块化设计
- **POX**: Nicira Networks (现VMware) 开源, 早期Python编写的OpenFlow控制器, 常用于教育与实验环境, 代码简洁易修改
- **Faucet**: ONF支持的开源项目, 基于OpenFlow的生产级控制器, 专为简化园区网部署设计, 支持YAML配置与自动化
- **Open vSwitch (OVS) 控制器**: Linux基金会支持的OVS项目, 内置于OVS的简易控制器, 适用于基础OpenFlow管理, 常与其他控制器协同工作

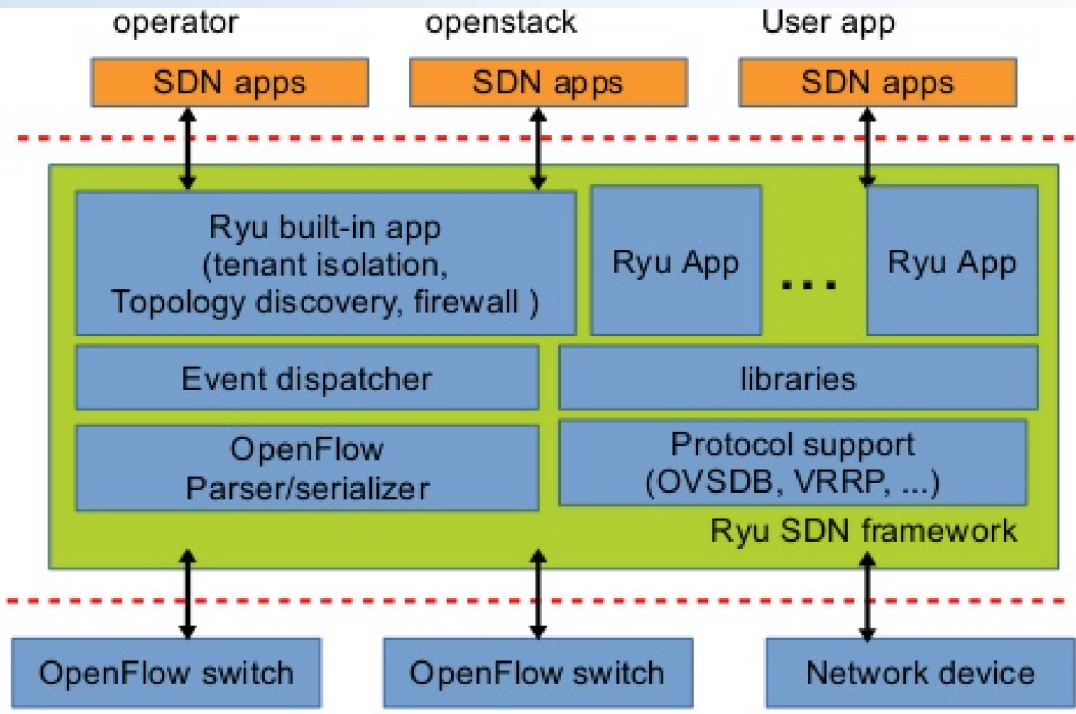
OpenDaylight总体架构



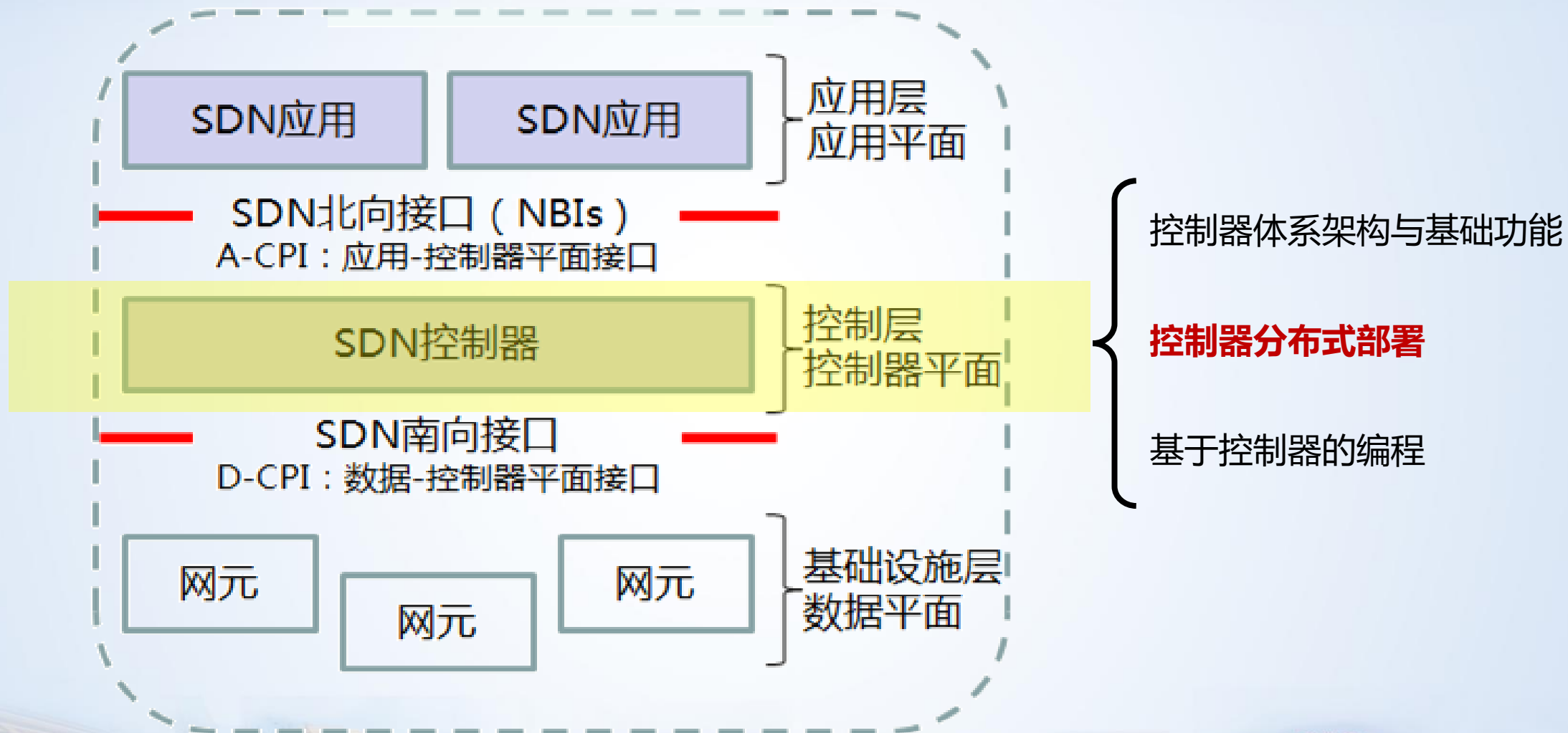
VTN：虚拟租户网络
DOVE：分布式覆盖虚拟以太网
DDoS：分布式拒绝服务
LISP：位置/身份标识分离协议
OVSDb：Open vSwitch数据库协议
BGP：边界网关协议
PCEP：路径计算单元协议
SNMP：简单网络管理协议



模块名	功 能
SAL: 服务抽象层	控制器模块化设计的核心，支持多种南向接口协议，屏蔽了协议间差异，为上层模块和应用提供一致性的服务
拓扑管理	负责管理节点、连接、主机等信息，并负责拓扑计算
统计管理	负责统计各种状态信息
主机追踪	负责追踪主机信息，记录主机的IP地址、MAC地址、VLAN以及连接交换机的节点和端口信息。该模块支持ARP请求发送及ARP消息监听，支持北向接口的主机创建、删除及查询
FRM: 转发规则管理	负责管理流规则的增加、删除、更新、查询等操作，并在内存数据库中维护所有安装到网络节点的流规则信息，当流规则发生变化时负责维护规则的一致性
交换机管理	负责维护网络中节点、节点连接器、接入点属性、三层配置、Span配置、节点配置、网络设备标识
ARP处理	负责处理ARP报文



模块名	功 能
OpenFlow 解析器	接收交换机报文时，解析模块从字节流中提取版本号、消息类型等头部信息并转换为Python对象；发送控制指令时，序列化模块将对象编码为符合OpenFlow规范的字节流
辅助协议 支持	支持NETCONF、OVSDb等配置协议
事件 调度器	基于eventlet库实现异步事件驱动模型，每个应用（RyuApp）启动时自动创建事件循环线程，通过队列接收事件（如Packet-In、Port-Status）并触发注册的处理函数
库	RYU的lib目录提供基础工具库，支撑核心功能： • 数据结构：定义dpid（交换机ID）、MAC/IP地址等网络实体标识，确保数据一致性 • 协议解析：packet子目录包含ICMP、MPLS等协议的解析逻辑 • 工具模块：ovs和netconf子目录提供与Open vSwitch、NETCONF设备交互的专用数据类型与接口
Ryu内置 APP	RYU内置多种开箱即用的应用，覆盖基础网络功能，如二层交换机、流表管理、拓扑发现等
Ryu APP	用户自定义应用是RYU灵活性的核心，基于RyuApp基类开发



控制器从集中式到分布式的演进

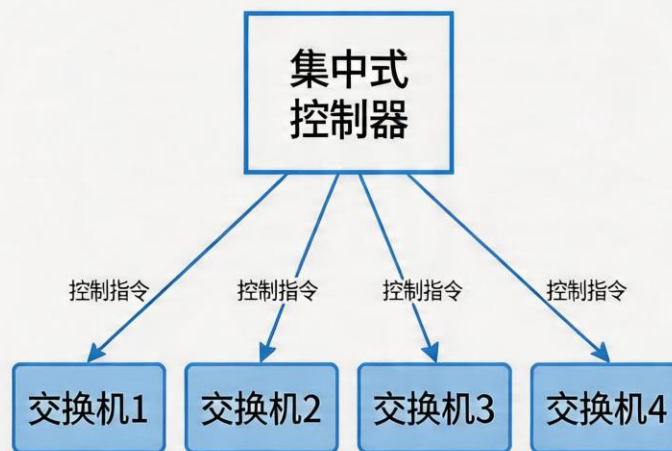
1. 集中式架构的局限性

- **单点故障 (SPOF)**: 控制器宕机导致全网瘫痪
- **性能瓶颈**: 无法处理超大规模流表请求
- **可扩展性**: 垂直扩展 (单机堆硬件) 成本高且存在物理极限

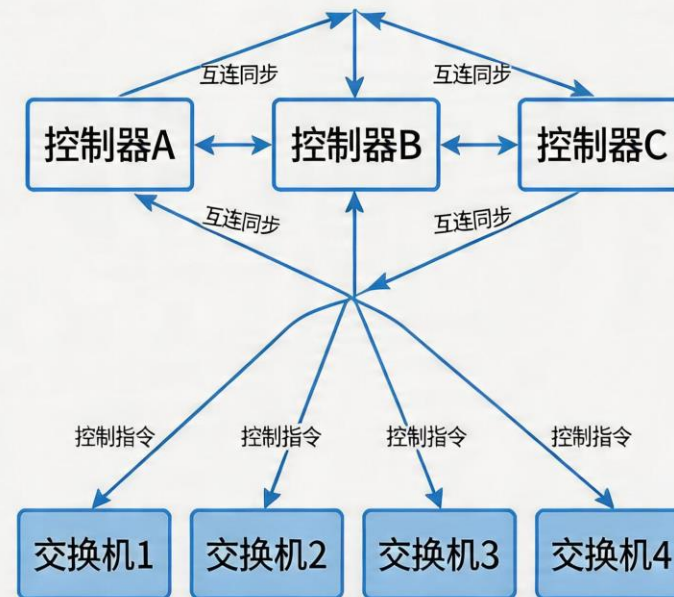
2. 分布式集群的核心价值

- **高可用性**: 节点冗余与故障状态快速同步
- **水平扩展**: 算力随节点增加线性增长
- **强一致性**: 依赖 Raft/Paxos 协议维护全局视图

集中式SDN控制器架构



分布式SDN控制器架构



分布式控制器的核心挑战：一致性

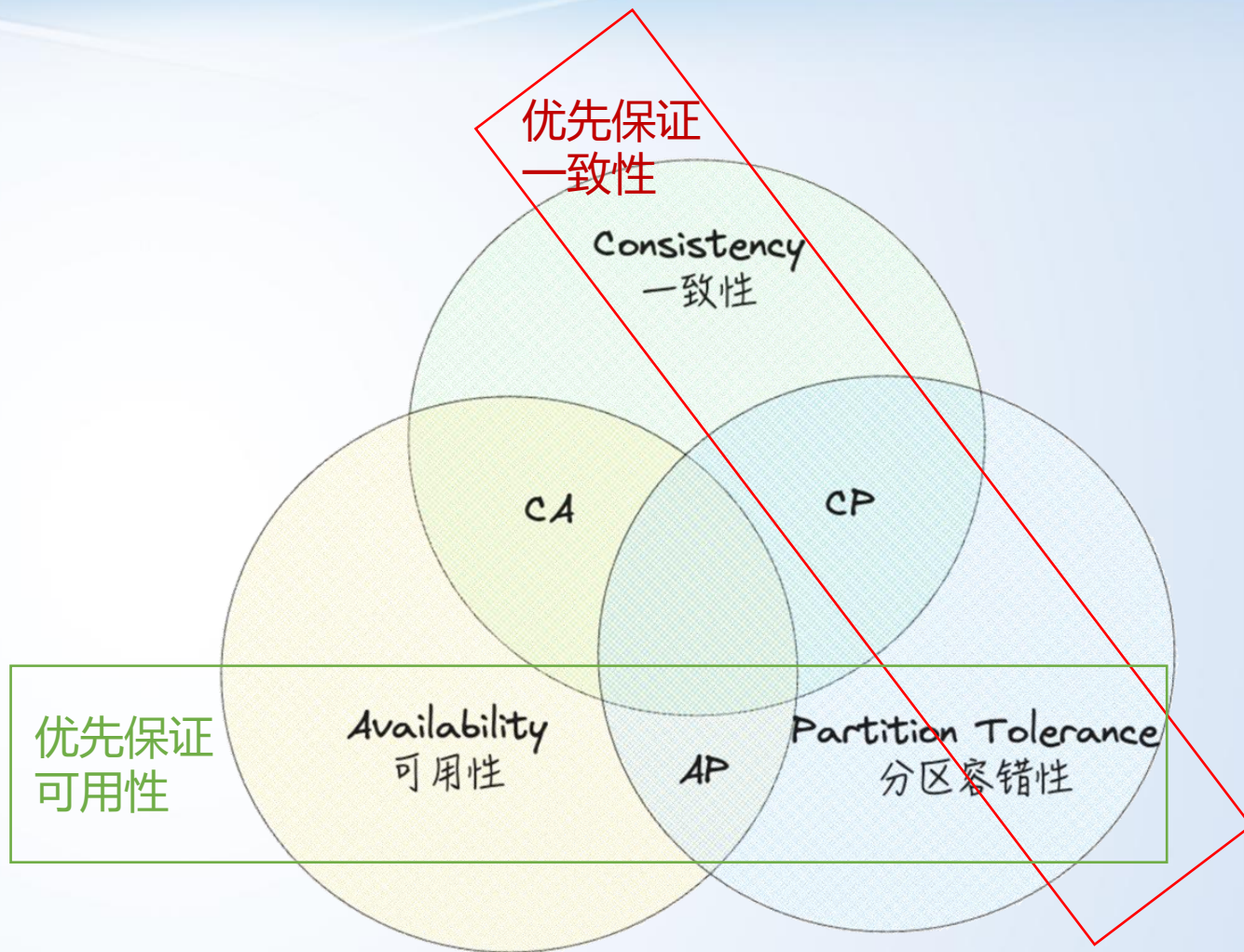


- **CAP 定理**：分布式系统的理论基石

- ✓ C (一致性)：所有节点在同一时间看到的数据完全相同。客户端向系统写入数据后，之后的任何读操作都会立刻读取到最新的写入结果
- ✓ A (可用性)：系统始终能提供服务，每个请求都能收到非错误的响应，但系统不保证能读到最新写入的数据
- ✓ P (分区容错性)：当集群中的部分节点之间网络不通（发生网络分区）时，系统整体仍能继续正常工作

- **一致性模型**：强一致性与最终一致性的权衡

- **东西向接口**：集群状态共享的核心纽带



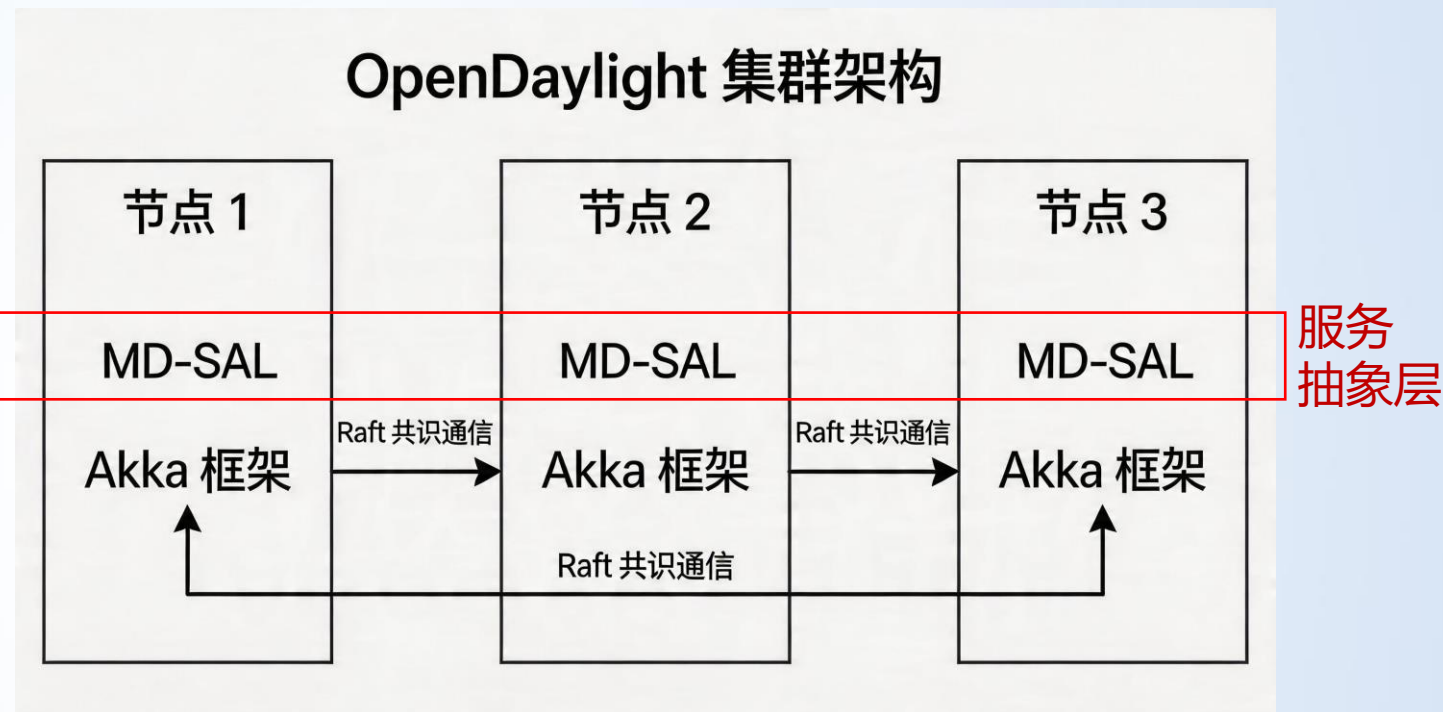
东西向接口定义了控制器集群内部的通信协议，是维护全局网络视图一致性的技术核心

MD-SAL 与 Akka 核心

- 将 MD-SAL 数据存储扩展为分布式系统
- 基于 Actor 模型实现节点间消息传递
- 统一管理集群状态同步与 RPC 调用

Raft 一致性与数据分片

- Raft 算法确保分片数据的强一致性
- Sharding 机制实现集群负载均衡

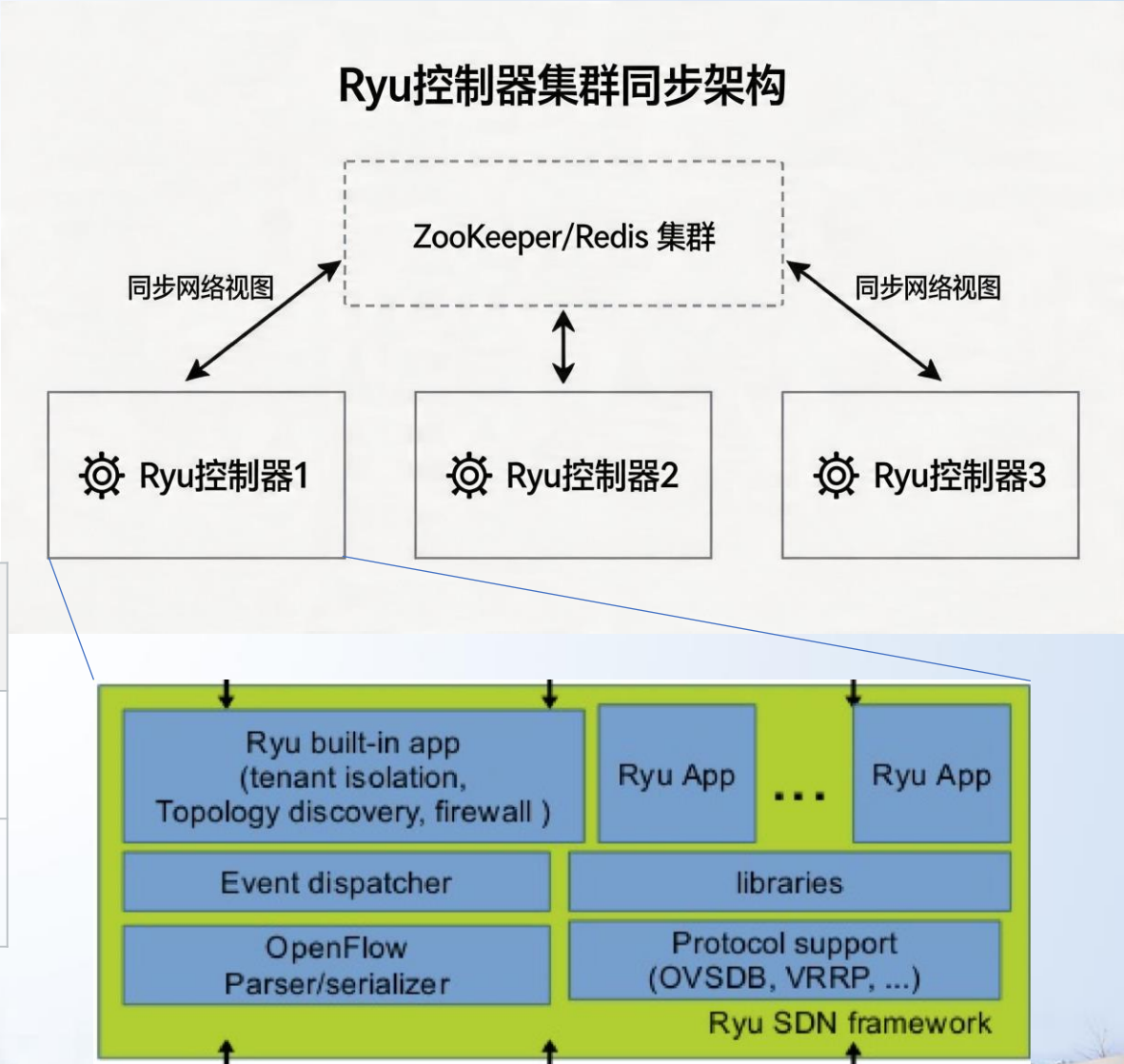


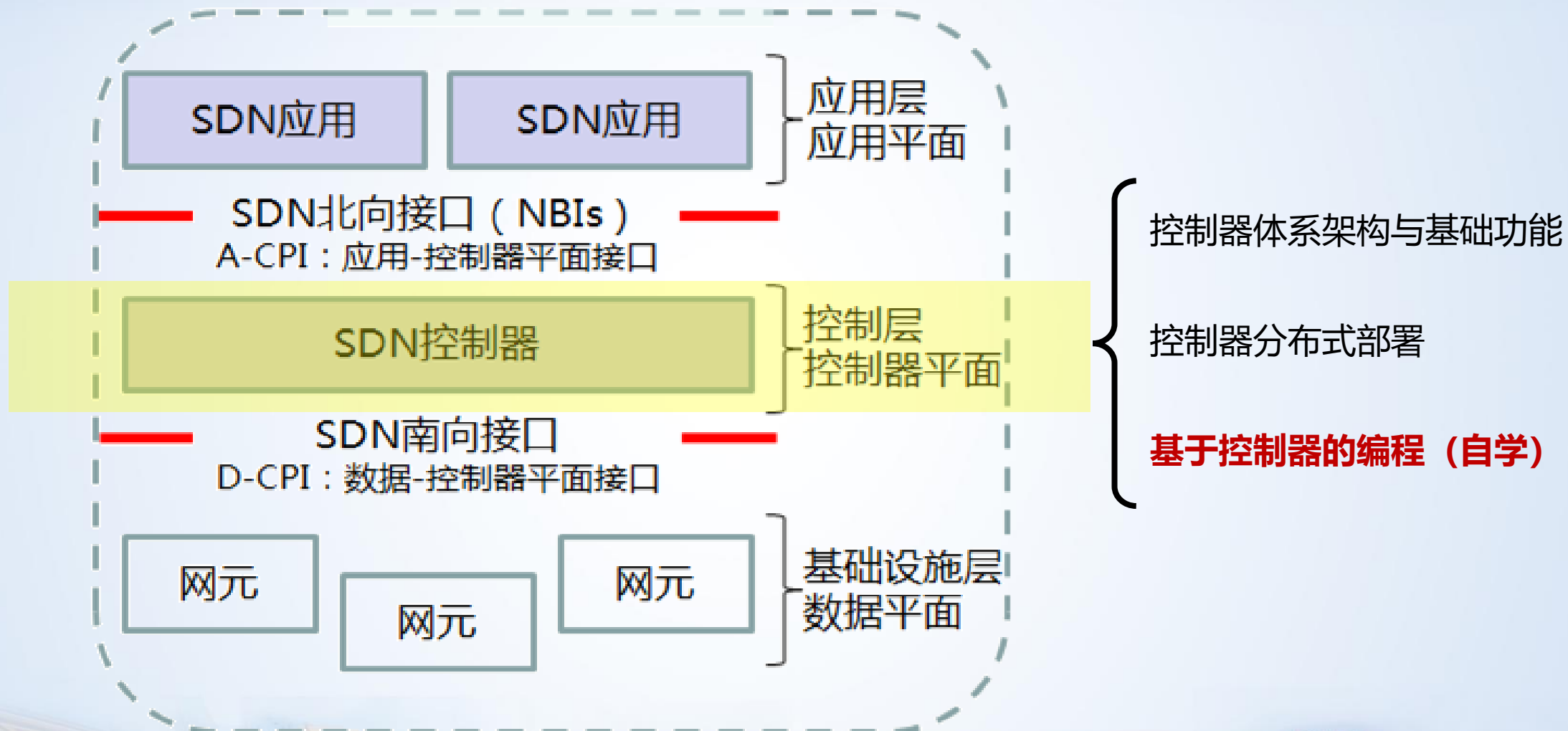
三节点 ODL 集群架构及 Raft 同步逻辑

Akka管通信、Raft管共识

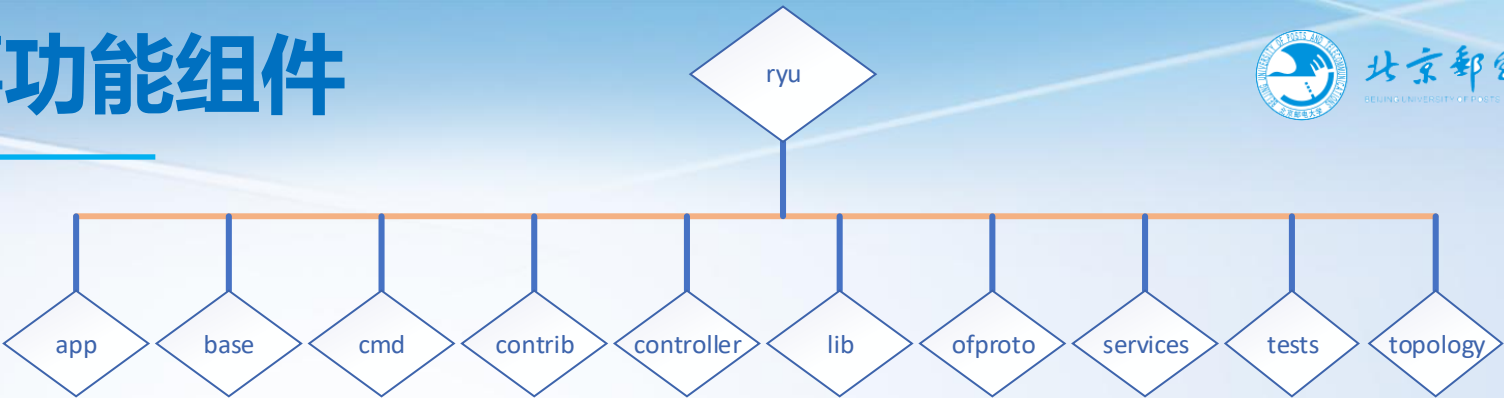
- 架构特性: 基于 Python, 高度模块化且灵活
- 瓶颈挑战: 原生不支持集群, 需解决单点故障
- 协调系统: 引入 Zookeeper 或 Redis 维护一致性
- 同步机制: 通过消息队列同步网络拓扑与状态信息

比较维度	OpenDaylight (ODL)	Ryu 扩展方案
集群架构	重量级 / 原生内置 (Akka)	轻量级 / 外部扩展
一致性维护	内部 Raft 协议同步	依赖分布式协调器 (ZK/Redis)





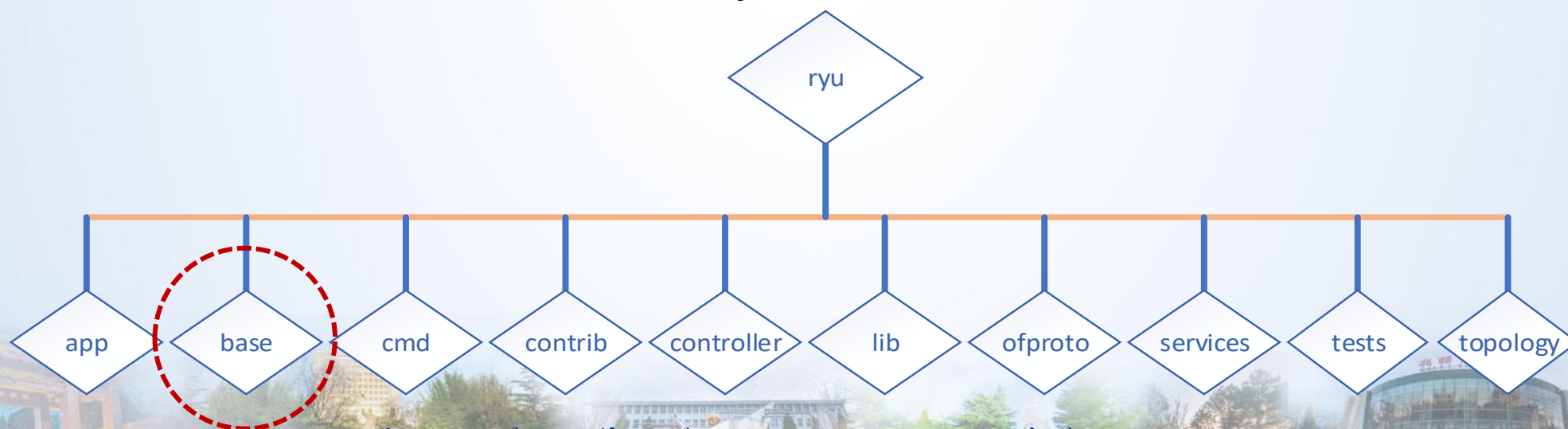
Ryu的目录结构及主要功能组件



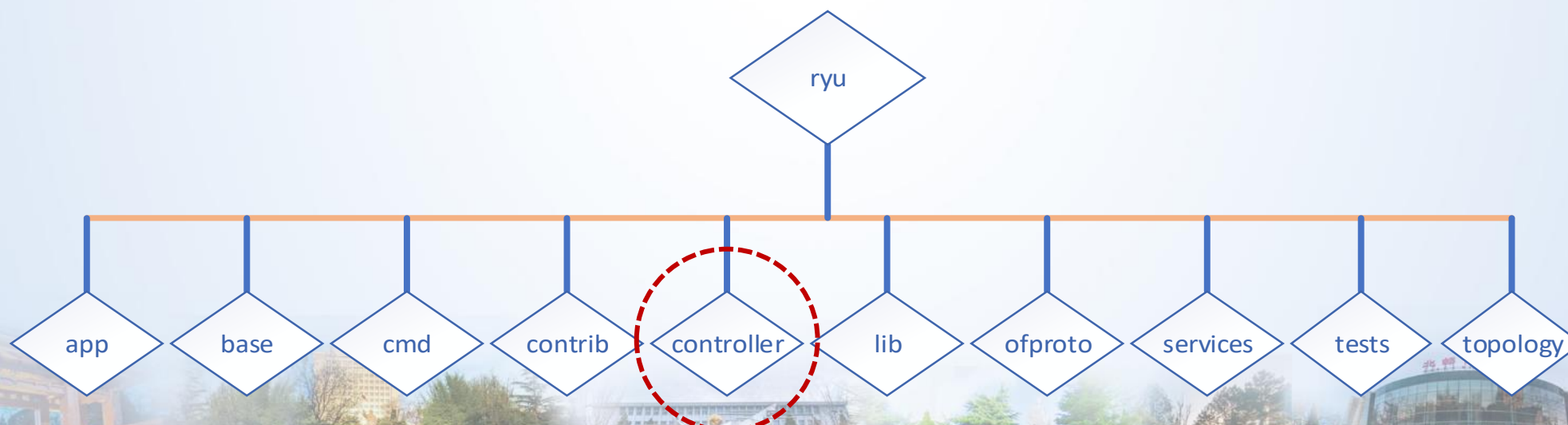
- Ryu是基于组件的软件定义网络的框架
- Ryu为软件组件提供定义良好的API，使开发人员可以轻松创建新的网络管理和控制应用程序

组件类型及说明	组件名	功能
基本组件	ryu/base/app_manager.py	提供了对其他组件的管理,被 ryu-manager 自动调用
	ryu/controller/dpset.py	管理 OF 交换机的组件,今后的版本中可能会被 ryu/topology 代替
	ryu/controller/ofp_handler.py	对控制器、交换机间握手、协商过程的处理
	ryu/controller/ofp_event.py	完成 OF 消息-事件的转化,提供北向接口 API
	ryu/controller/controller.py	控制器组件,管理与 OF 交换机连接的安全通道,接收 OF 消息,调用 ofp_event,并发布相应的“事件”,以触发订阅了该“事件”的组件的处理逻辑
与 Of 协议相关的组件	Ofproto1x.py(x=0/1/2/3)	定义了相应协议版本的参数
	Ofproto1x_parser.py(x=0/1/2/3)	定义了相关协议版本消息的封装格式
应用类组件	ryu/app/simple_switch.py	传统的 2 层交换机策略
	ryu/app/simple_switch_stp.py	无广播环路的 2 层交换机策略
	ryu/app/simple_isolation.py	基于 MAC 的过滤策略
	ryu/app/simple_vlan.py	实现了基本的 VLAN
	ryu/app/gre_tunnel.py	实现了多种隧道策略
	ryu/topology.py	交换机与链路状态监测组件,用于拓扑图的构建
基于 REST API 的应用组件	ryu/app/ofctl_rest.py	用于管理 OVS 中的流表
	ryu/app/rest_conf_switch.py	用于配置 OF 交换机
	ryu/app/rest_quantum.py	与 Quantum 的通信接口
	ryu/app/rest_router.py	路由策略
	ryu/app/rest_topology.py	用于对拓扑的管理
	ryu/app/rest_tunnel.py	隧道策略

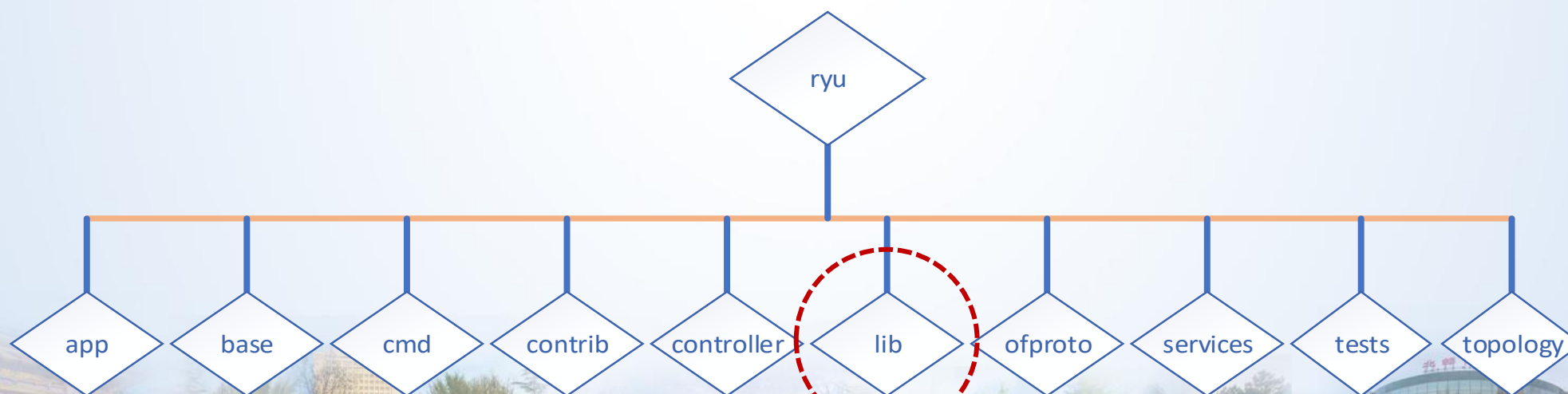
- app_manager.py的作用是RYU应用的管理中心，用于加载RYU应用程序，接受从APP发送过来的信息，同时也完成消息的路由
- 主要的函数有APP注册、注销、查找
- 定义了**RyuApp基类**
 - 定义基本属性，如name, threads, events, event_handlers, observers
 - 定义基本函数，如start(), stop()



- controller文件夹包含events.py, ofp_handler.py, **controller.py**等文件
- controller.py中定义了**OpenFlowController基类**，用于定义OpenFlow的控制器，用于处理交换机和控制器的连接等事件，同时还可以产生事件和路由事件；其事件系统的定义，可以查看events.py和ofp_events.py



- lib中定义了我们使用到的基本的数据结构，如dpid, mac和ip等数据结构
- 在lib/packet目录下，还定义了许多网络协议，如ICMP, DHCP, MPLS和IGMP等协议内容，而每一个数据包的类中都有parser和serialize两个函数。用于解析和序列化数据包



```
from ryu.base import app_manager
from ryu.controller import ofp_event
from ryu.controller.handler import MAIN_DISPATCHER, set_ev_cls
from ryu.ofproto import ofproto_v1_3

class MyRyuApp(app_manager.RyuApp):
    OFP_VERSIONS = [ofproto_v1_3.OFP_VERSION] # 指定支持的OpenFlow版本

    def __init__(self, *args, **kwargs):
        super(MyRyuApp, self).__init__(*args, **kwargs)
        # 初始化自定义变量
        self.mac_to_port = {}

    @set_ev_cls(ofp_event.EventOFPSwitchFeatures, MAIN_DISPATCHER)
    def switch_features_handler(self, ev):
        # 处理Switch Features事件
        datapath = ev.msg.datapath
        self.logger.info('Switch connected: %s', datapath.id)
```

- 类必须继承 `app_manager.RyuApp`
- 类属性 `OFP_VERSIONS` 指定应用支持的 OpenFlow 协议版本（可同时支持多个版本）
- `__init__` 中调用父类构造函数，并初始化应用所需的数据结构
- 使用 `@set_ev_cls` 装饰器注册事件处理函数，参数为事件类和交换机状态

```
@set_ev_cls(ofp_event.EventOFPSwitchFeatures, MAIN_DISPATCHER)
def switch_features_handler(self, ev):
    # 处理Switch Features事件
    datapath = ev.msg.datapath
    self.logger.info('Switch connected: %s', datapath.id)
```

➤ Ryu将OpenFlow消息以及内部状态变化封装为事件对象，通过set_ev_cls将处理函数绑定到特定事件

➤ 常用事件

- ofp_event.EventOFPPacketIn —— 收到 Packet-In 消息
- ofp_event.EventOFPSwitchFeatures —— 交换机特性协商完成
- ofp_event.EventOFPPortStatus —— 端口状态变化
- ofp_event.EventOFPFlowRemoved —— 流表项被移除

➤ 调度器状态（指定处理函数仅在特定连接状态下执行）

- handler.MAIN_DISPATCHER —— 正常交换状态（握手完成）
- handler.HANDSHAKE_DISPATCHER —— 握手阶段
- handler.CONFIG_DISPATCHER —— 特性请求/回复阶段
- 若不加状态，则默认在所有状态下都会调用

向指定的OpenFlow交换机下发一条优先级为1的流表项, 匹配所有目的MAC地址为00:11:22:33:44:55的数据包, 并将这些数据包交给交换机的传统处理模块 (OFPP_NORMAL) 转发

```
def send_flow_mod(self, datapath):  
    ofproto = datapath.ofproto  
    parser = datapath.ofproto_parser  
  
    # 构造流表修改消息 (如添加流表)  
    match = parser.OFPMatch(eth_dst='00:11:22:33:44:55')  
    actions = [parser.OFPActionOutput(ofproto.OFPP_NORMAL)]  
    inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS, actions)]  
    mod = parser.OFPFlowMod(datapath=datapath, priority=1,  
                             match=match, instructions=inst)  
    datapath.send_msg(mod)
```

- datapath.ofproto 和 datapath.ofproto_parser 分别获取当前协议版本的常量及解析器
- datapath.send_msg(msg) 将消息发送到交换机

一个完整的示例



```
# 导入 Ryu 应用基类, 所有 Ryu 应用都必须继承 app_manager.RyuApp
from ryu.base import app_manager

# 导入 OpenFlow 事件相关的模块, 用于处理交换机上报的事件
from ryu.controller import ofp_event

# 导入事件处理装饰器和交换机状态常量
# MAIN_DISPATCHER 表示交换机已完成握手, 处于正常转发状态
from ryu.controller.handler import MAIN_DISPATCHER, set_ev_cls

# 导入 OpenFlow 1.3 协议版本的定义
from ryu.ofproto import ofproto_v1_3

# 导入数据包解析库, 用于解析从交换机收到的原始数据包
from ryu.lib.packet import packet, ethernet
```

一个完整的示例



```
class SimpleSwitch13(app_manager.RyuApp):
    """
    一个简单的二层学习交换机应用, 支持 OpenFlow 1.3。
    它通过监听 Packet-In 事件学习 MAC 地址与端口的映射,
    并下发流表以避免后续数据包再触发 Packet-In。
    """
    # 指定此应用支持的 OpenFlow 协议版本为 1.3
    OFP_VERSIONS = [ofproto_v1_3.OFP_VERSION]

    def __init__(self, *args, **kwargs):
        """
        构造函数, 初始化父类并创建一个用于存储 MAC 地址学习结果的字典。
        """
        super(SimpleSwitch13, self).__init__(*args, **kwargs)
        # mac_to_port 是一个两层字典:
        # 第一层键为交换机 dpid (数据路径 ID),
        # 第二层键为 MAC 地址, 值为该 MAC 对应的交换机端口。
        self.mac_to_port = {}
```


一个完整的示例

```
class SimpleSwitch13(app_manager.RyuApp):  
  
    ...  
  
    @set_ev_cls(ofp_event.EventOFPSwitchFeatures, MAIN_DISPATCHER)  
    def switch_features_handler(self, ev):  
        """  
        处理交换机特性 (Switch Features) 事件。  
        当交换机与控制器握手成功并发送 Features Reply 时触发。  
        通常用于获取交换机的基本信息，这里只是记录交换机已连接。  
        """  
        # 从事件消息中获取 datapath 对象 (代表与交换机的连接)  
        datapath = ev.msg.datapath  
        # 记录交换机连接的日志，显示其 dpid  
        self.logger.info('Switch %s connected', datapath.id)
```

一个完整的示例



```
class SimpleSwitch13(app_manager.RyuApp):

    ...

    @set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
    def packet_in_handler(self, ev):
        """
        处理 Packet-In 事件。
        当交换机收到一个无法匹配任何流表项的数据包时,
        会将其封装为 Packet-In 消息发送给控制器。
        该方法实现了 MAC 地址学习并根据学习结果决定是精确转发还是泛洪。
        """
        # 获取消息对象
        msg = ev.msg
        # 获取 datapath 对象
        datapath = msg.datapath
        # 获取当前 OpenFlow 协议版本的常量模块和解析器模块
        ofproto = datapath.ofproto
        parser = datapath.ofproto_parser

        # 从消息的匹配字段中提取数据包进入交换机的端口号
        # 注意: 在 OpenFlow 1.3 中, Packet-In 消息的 match 字段包含 in_port
        in_port = msg.match['in_port']
```

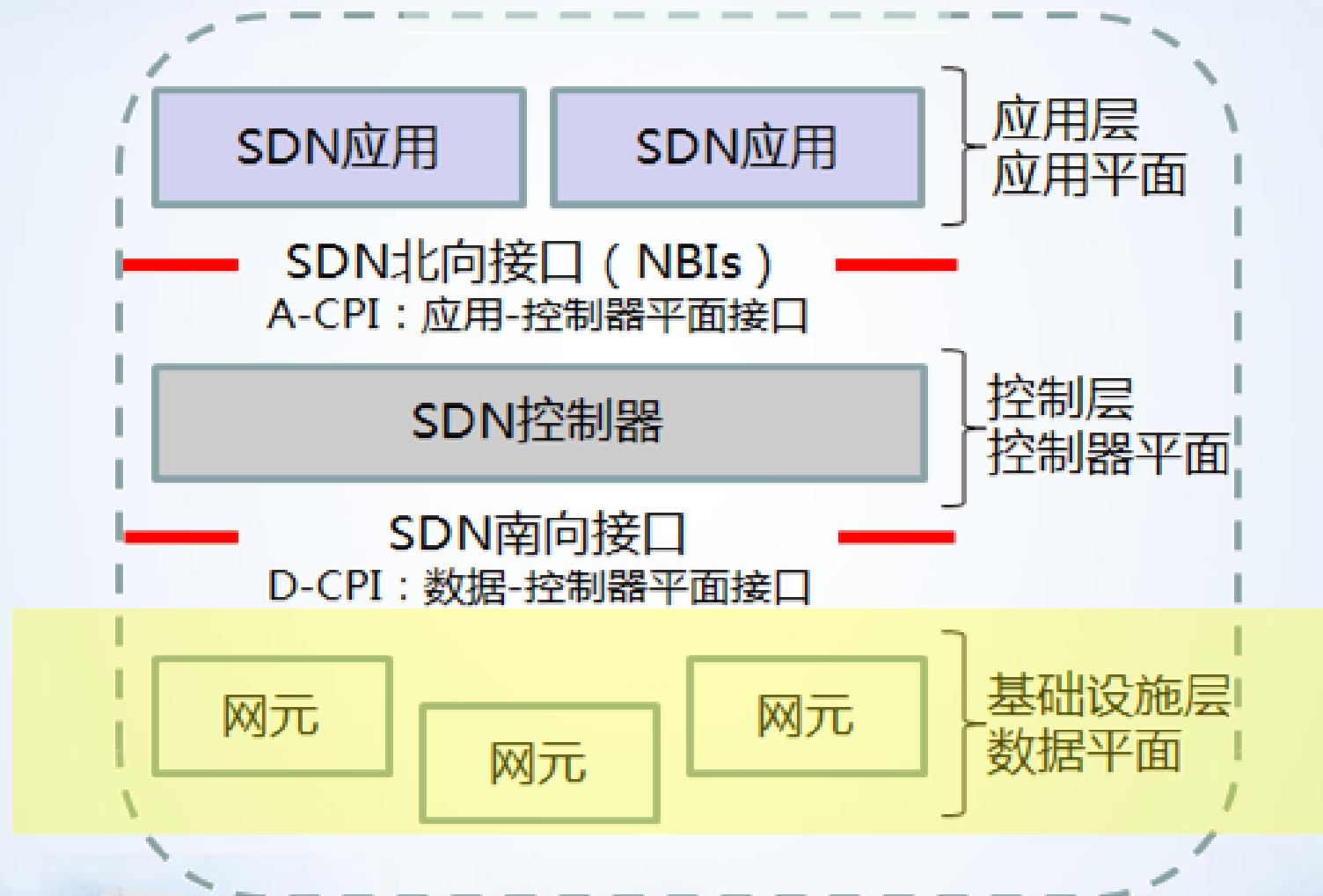
一个完整的示例



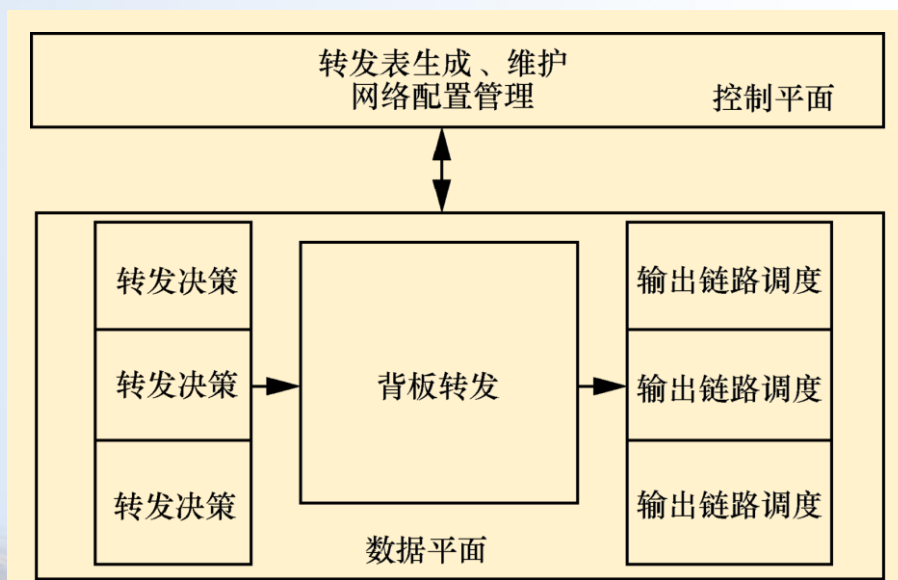
```
class SimpleSwitch13(app_manager.RyuApp):
    """
    一个简单的二层学习交换机应用, 支持 OpenFlow 1.3。
    它通过监听 Packet-In 事件学习 MAC 地址与端口的映射,
    并下发流表以避免后续数据包再触发 Packet-In。
    """

    @set_ev_cls(ofp_event.EventOFPSwitchFeatures, MAIN_DISPATCHER)
    def switch_features_handler(self, ev):
        """
        处理交换机特性 (Switch Features) 事件。
        当交换机与控制器握手成功并发送 Features Reply 时触发。
        通常用于获取交换机的基本信息, 这里只是记录交换机已连接。
        """
        # 从事件消息中获取 datapath 对象 (代表与交换机的连接)
        datapath = ev.msg.datapath
        # 记录交换机连接的日志, 显示其 dpid
        self.logger.info('Switch %s connected', datapath.id)

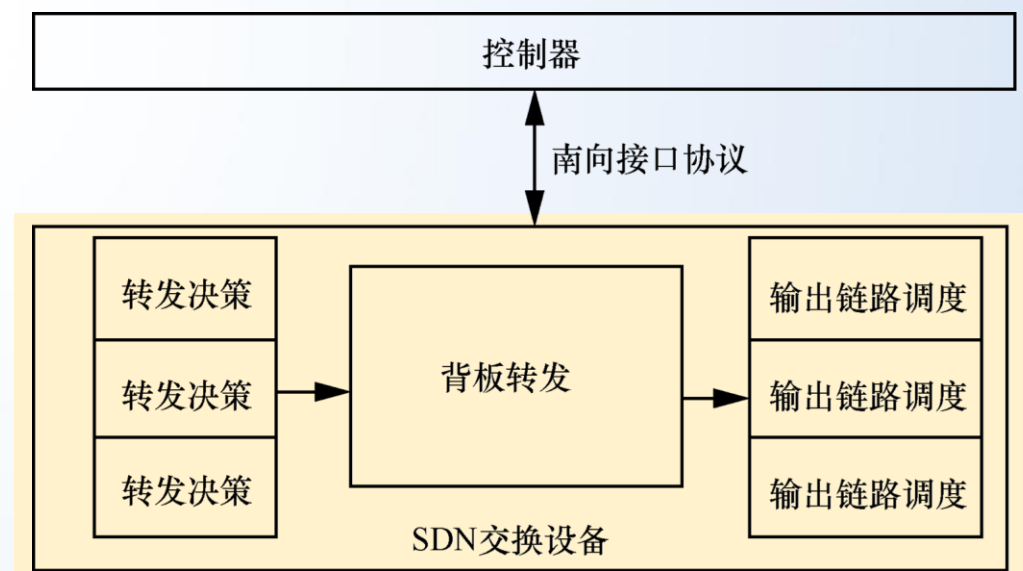
    @set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
    def packet_in_handler(self, ev):
        """
        处理 Packet-In 事件。
        """
```

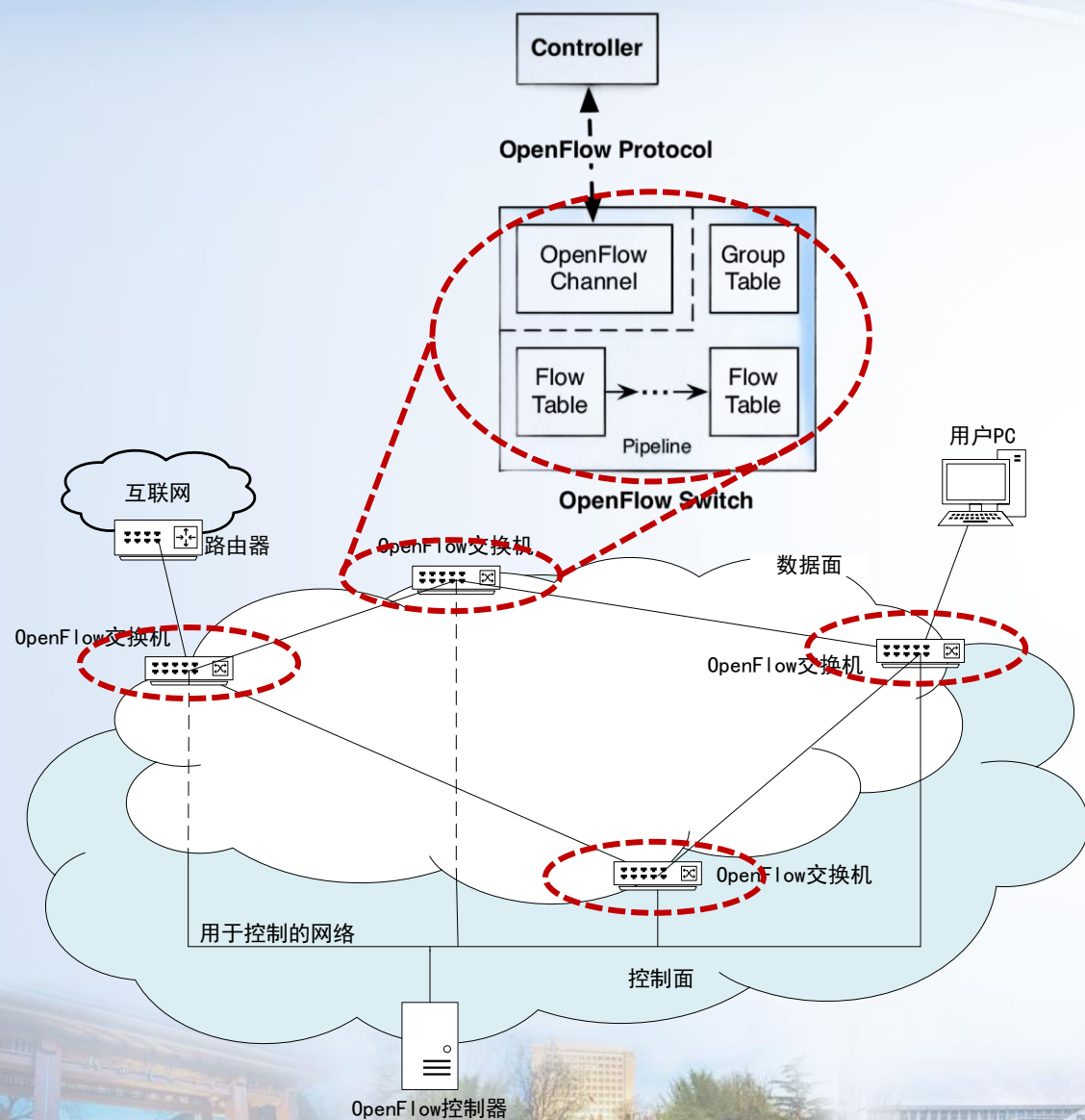
- SDN交换设备**没有**二层交换机、路由器、三层交换机之分，基本功能包括**转发决策**、**背板转发**、**输出链路调度**
 - 转发决策：用流表代替了传统网络设备二层和三层转发表，该流表中的每个表项都代表了一种流解析以及相应的处理动作。数据分组进入SDN交换机后，先与流表进行匹配查找，若与其中一个表项匹配成功，则执行相应的处理动作；若无匹配项则上交控制器，由其决定处理决策
 - 背板转发：通过背板把各端口连接起来，数据分组经转发决策后，经背板从入端口转发到出端口
 - 输出链路调度：SDN交换机对QoS的支持主要有基于流表项设置报文入队列、根据Meter进行限速、基于Counter进行计费、基于Group的Select功能进行队列调度等



传统交换设备

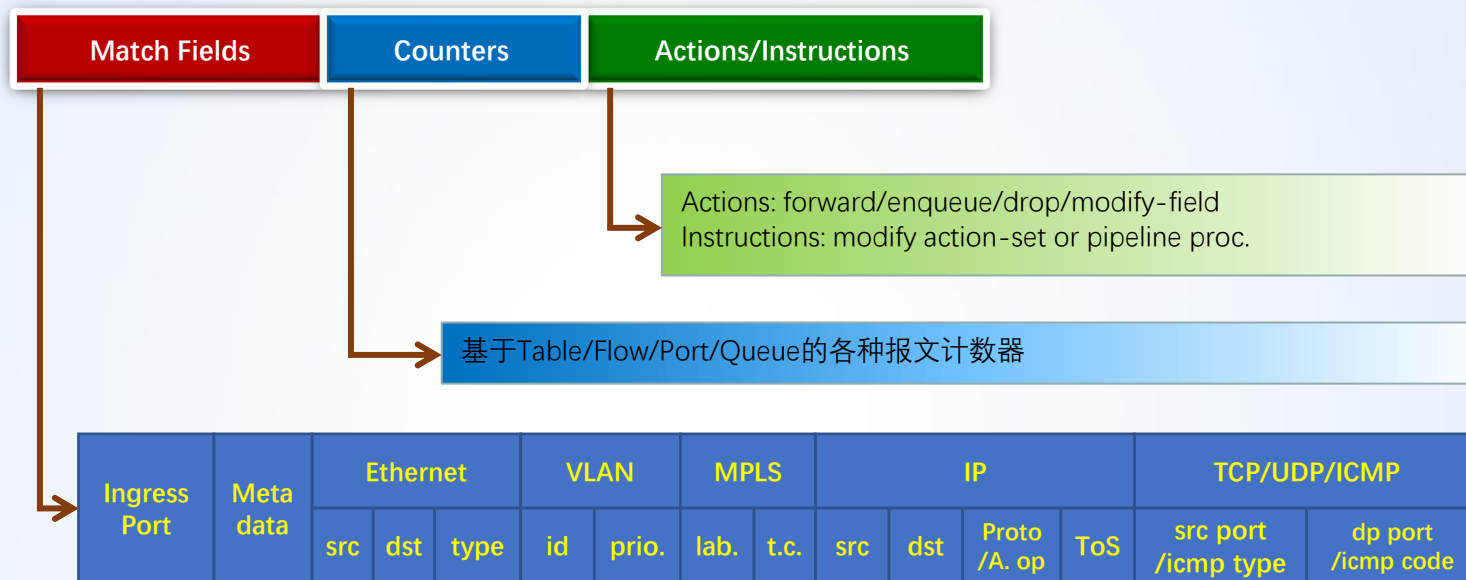


SDN交换设备



- **OpenFlow交换机**负责数据层的转发，主要由**OpenFlow表项**和**安全通道**组成，OpenFlow交换机与控制器之间通过安全通道进行消息交互，实现表项下发、状态上报等功能
 - ✓ OpenFlow表项为OpenFlow的关键组成部分，由控制器下发来实现控制平面对转发平面的控制
 - ✓ 安全通道是连接OpenFlow交换机到控制器的接口；控制器通过这个接口控制和管理交换机，同时控制器接收来自交换机的事件并向交换机发送数据包；交换机和控制器通过安全通道进行通信，而且所有的信息必须按照OpenFlow协议规定的格式来执行
- OpenFlow交换机有两种
 - ✓ OpenFlow-Only Switch：仅支持OpenFlow转发
 - ✓ OpenFlow-Hybrid Switch：既支持OpenFlow转发，也支持普通二三层转发

- 流表是OpenFlow对网络设备数据转发功能的抽象，表项包括网络中各个层次的配置信息

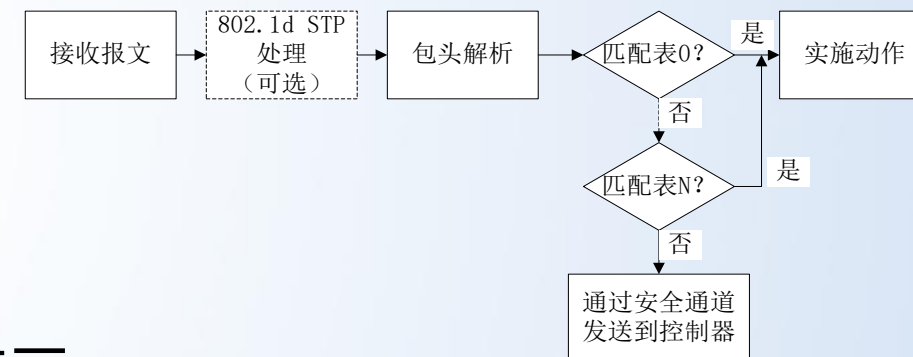


- **包头域/匹配域 (Match Fields)**：用于对交换机接收到的数据包的包头内容进行匹配
- **计数器 (Counters)**：用于统计数据流量相关信息，可以针对交换机中的每张流表、每个数据流、每个设备端口、每个转发队列进行维护
- **动作 (Actions/Instructions)**：用于指示交换机在收到匹配数据包后如何对其进行处理，决定了OpenFlow对转发面行为的抽象能力，OpenFlow交换机缺少控制平面的能力，因此需要用动作来详细说明交换机将要对数据包所做的处理

■包头域/匹配域用于匹配交换机接收到的数据包的包头内容

- OpenFlow 1.0包头域包含12个元组 (tuple)
- 涵盖OSI网络模型中第二至第四层的网络配置信息
- 每一个元组中的数值可以是一个确定的值或者是 “ANY”

入端口	源MAC地址	目的MAC地址	以太网类型	VLAN ID	VLAN优先级	源IP地址	目的IP地址	IP协议	IP TOS 位	TCP/UDP 源端口	TCP/UDP 目的端口
Ingress Port	Ether Source	Ether Des	Ether Type	VLAN ID	VLAN Priority	IP Src	IP Dst	IP Proto	IP ToS bits	TCP/UDP Src Port	TCP/UDP Dst Port



用形如<header: actions, counters>的流表表示:

- 若输入分组匹配到特定的header, 计数器被更新, 相应的动作被执行
- 若分组匹配多个表项, 优先级最高的表项被选中 (编号越大, 优先级越高)
- 若输入分组没有匹配任何一个表项, 分组被转发给控制器

- 流表中包含的计数器用来统计已处理的数据包数量等信息。在OpenFlow 1.0中，有四种计数器：各流表的**Per Table计数器**，各物理端口的**Per Port计数器**，各流表项的**Per Flow计数器**，各队列的**Per Queue计数器**

Per Table（基于表）计数器

内容	比特数
Active Entries（有效表项）	32
Packet Lookups（查表的数据包数）	64
Packet Matches（匹配的数据包数）	64

Per Flow（基于流）计数器

内容	比特数
Received Packets（接收的数据包数）	64
Received Bytes（接收的字节数）	64
Duration(msec)（持续时间，毫秒）	32
Duration(usec)（持续时间，微秒）	32

Per Queue（基于队列）计数器

内容	比特数
Transmit Packets（发送的数据包数）	64
Transmit Bytes（发送的字节数）	64
Transmit Overrun Errors（发送超限错误）	64

Per Port（基于端口）计数器

内容	比特数
Received Packets（接收的数据包数）	64
Transmitted Packets（发送的数据包数）	64
Received Bytes（接收的字节数）	64
Transmitted Bytes（发送的字节数）	64
Receive Drops（丢弃的接收数据包）	64
Transmit Drops（丢弃的发送数据包）	64
Receive Errors（接收错误）	64
Transmit Errors（发送错误）	64
Receive Frame Alignment Errors（接收帧的字节定位错误）	64
Receive Overrun Errors（接收帧的超限错误）	64
Receive CRC Errors（接收帧CRC校验错误）	64
Collisions（冲突）	64

■用于指示交换机在收到匹配的数据包后应该如何对其进行处理

■必备动作 (Required Actions) vs. 可选动作 (Optional Actions)

– 必备动作：需要所有OpenFlow交换机默认支持

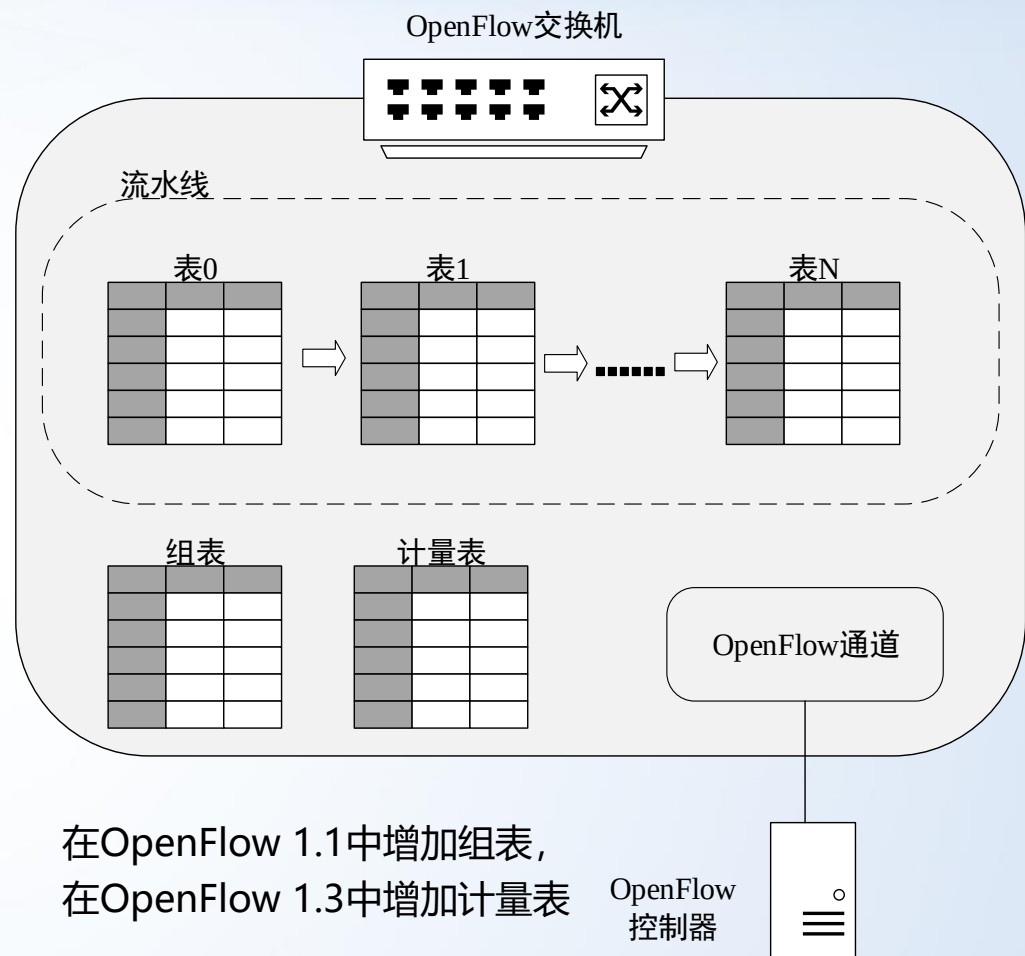
– 可选动作：需要交换机告知控制器它所能支持的行动作种类

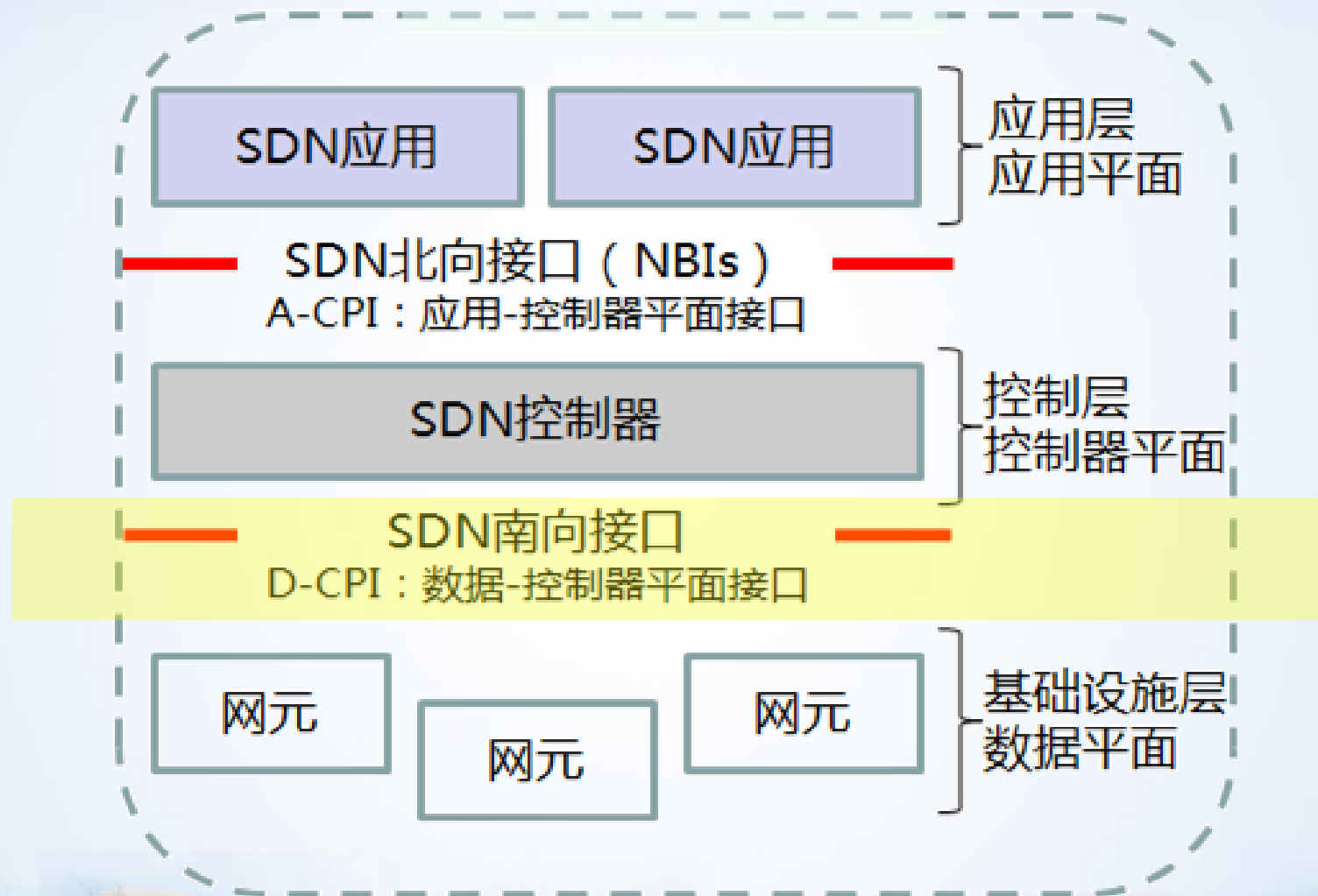
类型	名称	说明
必备动作	转发 (Forward)	交换机必须支持将数据包转发给设备的物理端口及如下的一个或多个虚拟端口，实现组播、多路径转发、负载均衡等功能： ➢ALL：转发给所有出端口，但不包括入端口 ➢CONTROLLER：封装数据包并转发给控制器 ➢LOCAL：转发给本地的网络栈 ➢TABLE：对packet_out消息执行流表的动作 ➢IN_PORT：从入端口发出
	丢弃 (Drop)	交换机对没有明确指明处理动作的流表项，将会对与其所匹配的所有数据包进行默认的丢弃处理
可选动作	转发 (Forward)	交换机可选支持将数据包转发给如下的虚拟端口： ➢NORMAL：利用交换机所能支持的传统转发机制（例如二层的MAC、VLAN信息或者三层的IP信息）处理数据包 ➢FLOOD：遵照最小生成树从设备出端口洪泛发出，但不包括入端口
	排队 (Enqueue)	交换机将数据包转发到某个出端口对应的转发队列中，便于提供QoS支持
	修改域 (Modify-Field)	交换机修改数据包的包头内容，各个字段值、封装/去封装，具体可以包括： ➢设置VLAN ID、VLAN优先级，剥离VLAN头 ➢修改源MAC地址、目的MAC地址 ➢修改源IPv4地址、目的IPv4地址、ToS位 ➢修改源TCP/IP端口、目的TCP/IP端口

OpenFlow流表示例

Ingress Port	Source MAC	Dest MAC	Ether Type	VLAN ID	VLAN Priority	IP SRC	IP DEST	IP Protocol	IP TOS	TCP/UDP SRC	TCP/UDP DEST		Action	Priority	Counter
*	3c:07:54:*	*	*	Switching	*	*	*	*	*	*	*		Fwd Port 10	100	
*	*	*	Routing	*	*	*	192.168.1.*	*	*	*	*		Fwd Port 12	100	
Port 1	*	*	Replication/SPAN	*	*	*	*	*	*	*	*		Fwd Port 14...24	100	
*			Firewall/Security	*	*	*	*	*	*	*	25		Drop	100	
*	*	*	*	Inspection	*	*	*	0x0800	*	*	*		Controller	100	
*	00:01:E7:*	*	*	Vlan10	*	Combinations	*	*	*	*	80		Fwd Port 8	200	
*	*	*	Multi-action ; NAT	*	*	*	192.168.1.*	*	*	*	80		Rewrite 10.1.2.3, Fwd port 9	200	
			Local handling	*	*	*	10 *	*	*	*	*		Local	200	

- 从OpenFlow 1.1开始，添加了“组”这一抽象化概念。通过这种抽象化，可将多个端口作为“组”进行处理。组对应的动作可以通过“Group动作”指定对象组ID号来执行。Group动作与Output动作、Set VLAN ID动作相同，是动作类型之一
- 在OpenFlow 1.2及其以前的版本中，不能在OpenFlow的框架内直接对各流进行计量，所以OpenFlow并不能直接实现QoS功能。在OpenFlow 1.3中，增加了计量表用于流的计量。通过使用计量表对各流进行测量，可实现之前很难实现的基于OpenFlow的直接QoS功能





- 南向接口协议是控制器与交换机间信息交互的关键，其设计目标主要包括：
 - 1. 收集数据平面信息，下发控制策略，指导转发行为
 - 2. 实现网络配置与管理
 - 3. 执行路径计算，涵盖链路属性、状态和拓扑信息

南向接口协议		设计目标
控制类协议	OpenFlow	用于OpenFlow交换机与控制器的信息交互
配置类协议	OF-Config	用于OpenFlow交换机的配置与管理
	NETCONF	用于网络设备的配置与管理
	OVSDB	用于Open vSwitch的配置与管理
路径计算与 控制协议	PCEP	PCEP为PCE和PCC之间的通信协议，实现路径计算
	I2RS	I2RS体系架构中的南向接口协议
策略分发与 消息协议	OpFlex	思科ACI体系中的策略控制协议
	XMPP	用于即时通信、游戏平台、语音与视频会议系统， openContrail控制器利用xMPP与vRouter进行信息交互

Controller Application

Network OS

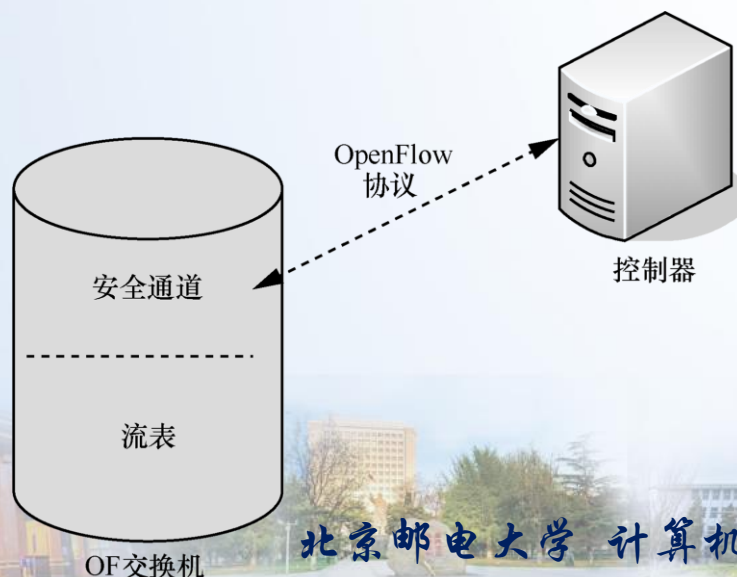
Events from switches

Topology changes,
Traffic statistics,
Arriving packets

Commands to switches

(Un)install rules,
Query statistics,
Send packets

可编程的控制器的工作机制

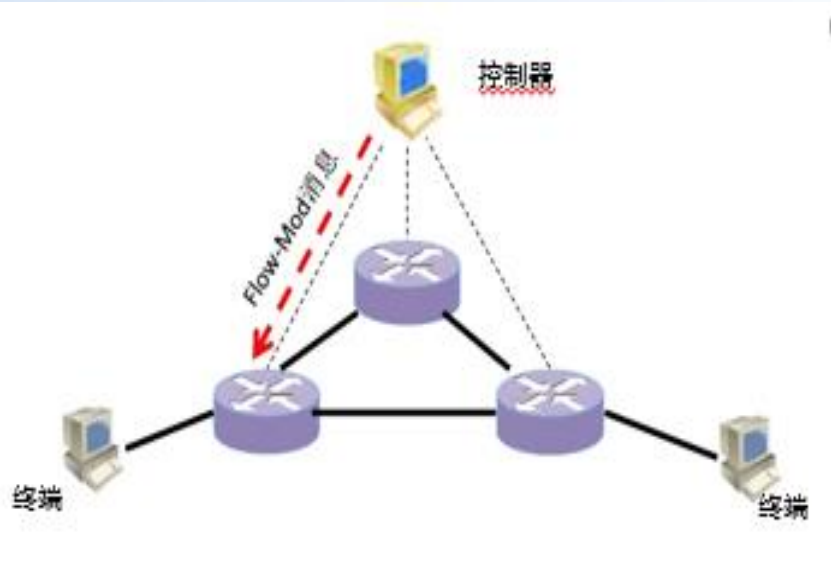


■ OpenFlow 协议是用来描述控制器和 OpenFlow 交换机之间数据交互所用信息的接口标准，其核心是 OpenFlow 协议信息的集合

■ OpenFlow 协议支持三种消息类型

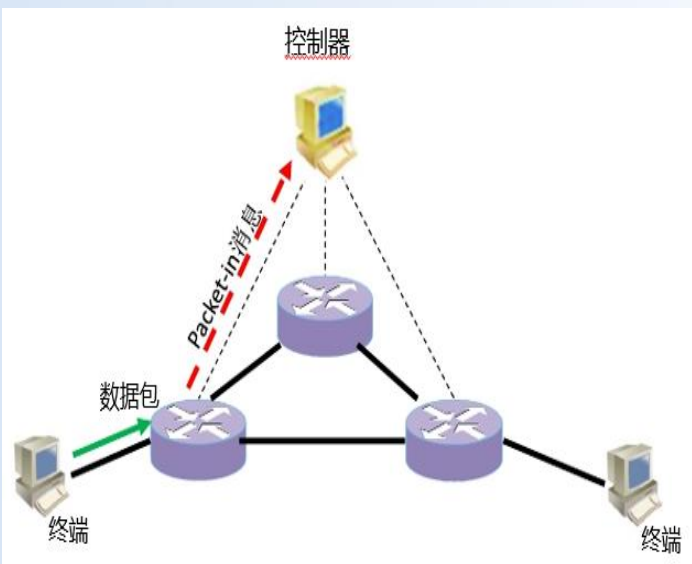
- **Controller-to-Switch (控制器—交换机)**：由控制器发起，用来管理或获取 OpenFlow 交换机的状态，包括 **Features**、**Modify-State**、**Read-State** 等
- **Asynchronous (异步)**：由 OpenFlow 交换机发起，用来将网络事件或交换机状态变化更新到控制器，如 **Packet-in**、**Flow-Removed**、**Port-Status** 和 **Error** 等
- **Symmetric (对称)**：双向对称消息，用于建立连接和检测对方是否在线，包括 **Hello** 和 **Echo**

■ Controller-to-Switch（控制器—交换机）：这类消息由控制器发起



子类型	功能描述
Features	安全通道建立以后，控制器会立即发送 Features-Request 消息给交换机，以获取交换机支持的相关特性。
Configuration	控制器可以通过Set-Config消息设置交换机的配置信息，通过Get-Config消息查询配置信息，交换机需要通过Config-Reply消息做出应答。
Modify-State	控制器通过Port-mod消息管理交换机的端口状态，通过 Flow-mod 消息增加或者删除交换机的流表项。
Read-State	OpenFlow会维护每张表、每个流表项、每个端口、每个队列相应的计数器，当控制器需要统计信息时，还会向交换机发送相关的Request消息，请求相关信息。
Send-Packet	很多情况下，控制器需要发送消息到数据平面，这时可以通过 Packet-out 消息封装好数据分组并传给OF交换机，并在该消息中指定特定的动作表，指导交换机如何处理这个数据分组，而不再进行流表的匹配。
Barrier	控制器通过Barrier-Request消息确保之前下发的消息已经被交换机执行完。

- Asynchronous（异步）：这类消息由交换机主动发起，用来将网络事件或交换机状态的变化更新到控制器



子类型	功能描述
Packet-in	收到的数据分组如果在流表中没有匹配的流表项，或者匹配的流表项中给出了转发动作，但动作中指定端口为CONTROLLER时，OF交换机会封装Packet-in消息，把这个数据分组上交给控制器。
Flow-Removed	交换机中的流表项因为超时或修改等原因被删除掉时，会触发Flow-Removed消息，触发的前提是在下发这条流表项时flags字段置位了OFPFF_SEND_FLOW_REM。
Port-Status	交换机端口状态发生变化时（如Up/Down），会触发Port-Status消息报告端口的信息以及发生的状态变化（如Add、Delete或者Modify）。
Error	交换机通过Error消息来通知控制器发生错误的信息。

- Symmetric（对称）：这类消息可由控制器发起，也可以由OF交换机发起

子类型	功能描述
Hello	认证通过后，双方通过握手消息（Hello）建立安全通道。
Echo	双方均可以主动向对方发出Echo-Request消息，接收者需要回复Echo-Reply。该消息用来“保活”，也可以用来测量延迟。
Vendor	Vendor是为未来的协议版本预留的，以便允许OF交换机厂商提供额外的OpenFlow功能。

□ Openflow协议数据包由OpenFlow Header和OpenFlow Message两部分组成

➤ OpenFlow Header的结构:

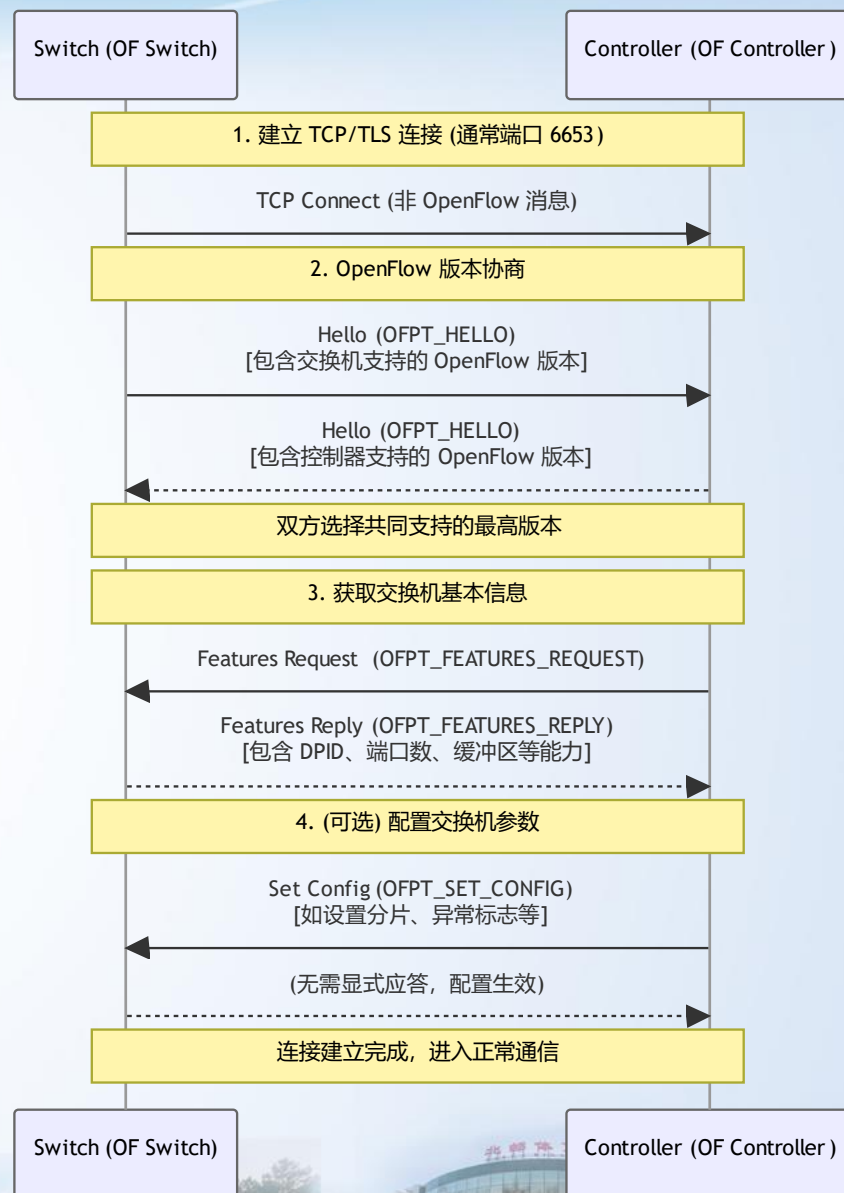
```
struct ofp_header {  
    uint8_t version;    /* OFP_VERSION. */  
    uint8_t type;        /* One of the OFPT_ constants. */  
    uint16_t length;     /* Length including this ofp_header. */  
    uint32_t xid;        /* Transaction id associated with this packet.  
                        Replies use the same id as was in the request  
                        to facilitate pairing. */  
};
```

➤ OpenFlow Message结构与具体消息类型有关

```
enum ofp_type {  
    /* Immutable messages. */  
    OFPT_HELLO,           /* Symmetric message */  
    OFPT_ERROR,           /* Symmetric message */  
    OFPT_ECHO_REQUEST,    /* Symmetric message */  
    OFPT_ECHO_REPLY,      /* Symmetric message */  
    OFPT_VENDOR,         /* Symmetric message */  
  
    /* Switch configuration messages. */  
    OFPT_FEATURES_REQUEST, /* Controller/switch message */  
    OFPT_FEATURES_REPLY,   /* Controller/switch message */  
    OFPT_GET_CONFIG_REQUEST, /* Controller/switch message */  
    OFPT_GET_CONFIG_REPLY,  /* Controller/switch message */  
    OFPT_SET_CONFIG,        /* Controller/switch message */  
  
    /* Asynchronous messages. */  
    OFPT_PACKET_IN,        /* Async message */  
    OFPT_FLOW_REMOVED,     /* Async message */  
    OFPT_PORT_STATUS,      /* Async message */  
  
    /* Controller command messages. */  
    OFPT_PACKET_OUT,       /* Controller/switch message */  
    OFPT_FLOW_MOD,         /* Controller/switch message */  
    OFPT_PORT_MOD,         /* Controller/switch message */  
  
    /* Statistics messages. */  
    OFPT_STATS_REQUEST,    /* Controller/switch message */  
    OFPT_STATS_REPLY,      /* Controller/switch message */  
  
    /* Barrier messages. */  
    OFPT_BARRIER_REQUEST, /* Controller/switch message */  
    OFPT_BARRIER_REPLY,   /* Controller/switch message */  
  
    /* Queue Configuration messages. */  
    OFPT_QUEUE_GET_CONFIG_REQUEST, /* Controller/switch message */  
    OFPT_QUEUE_GET_CONFIG_REPLY    /* Controller/switch message */  
};
```

OpenFlow协议消息流程 —— 建立连接

- TCP 连接建立（非 OpenFlow 消息）：交换机主动连接控制器
- 版本协商：通过 **Hello** 消息交换版本信息，确定共同支持的最高 OpenFlow 版本
- 能力获取：控制器发送 **Features Request**，交换机回复 **Features Reply**，提供自身基本信息（如 DPID、端口等）
- 可选配置：控制器可发送 **Set Config** 设置交换机行为（如分片策略），该消息通常无需显式应答
- 完成：之后双方可开始正常的异步消息和控制消息交互



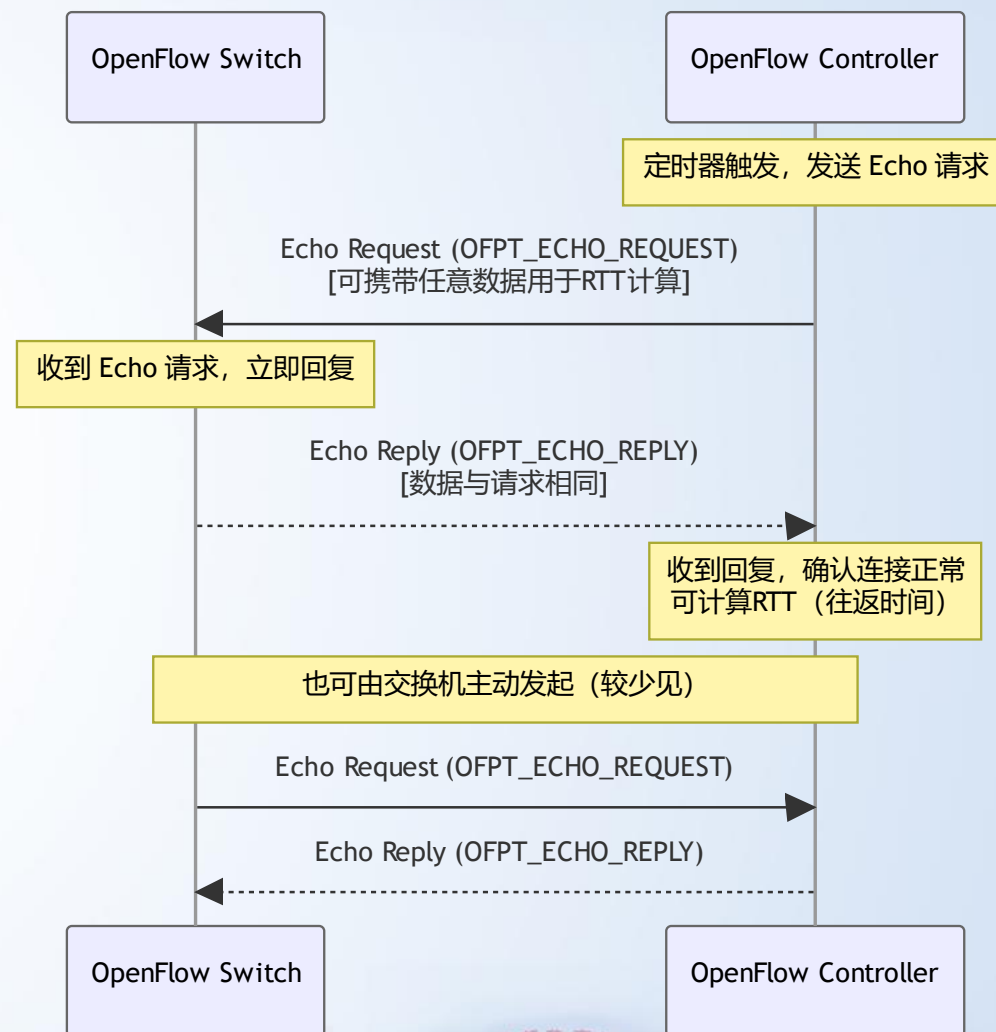
OpenFlow协议消息流程 —— 连接保活

■ 控制器主动发起（最常见场景）：

- 控制器定期向交换机发送 Echo Request 消息，可携带任意数据（如时间戳、序列号）
- 交换机收到后必须立即回复 Echo Reply，并将数据原样返回
- 控制器通过接收到的回复确认交换机在线，并可计算往返时间（RTT）用于网络诊断

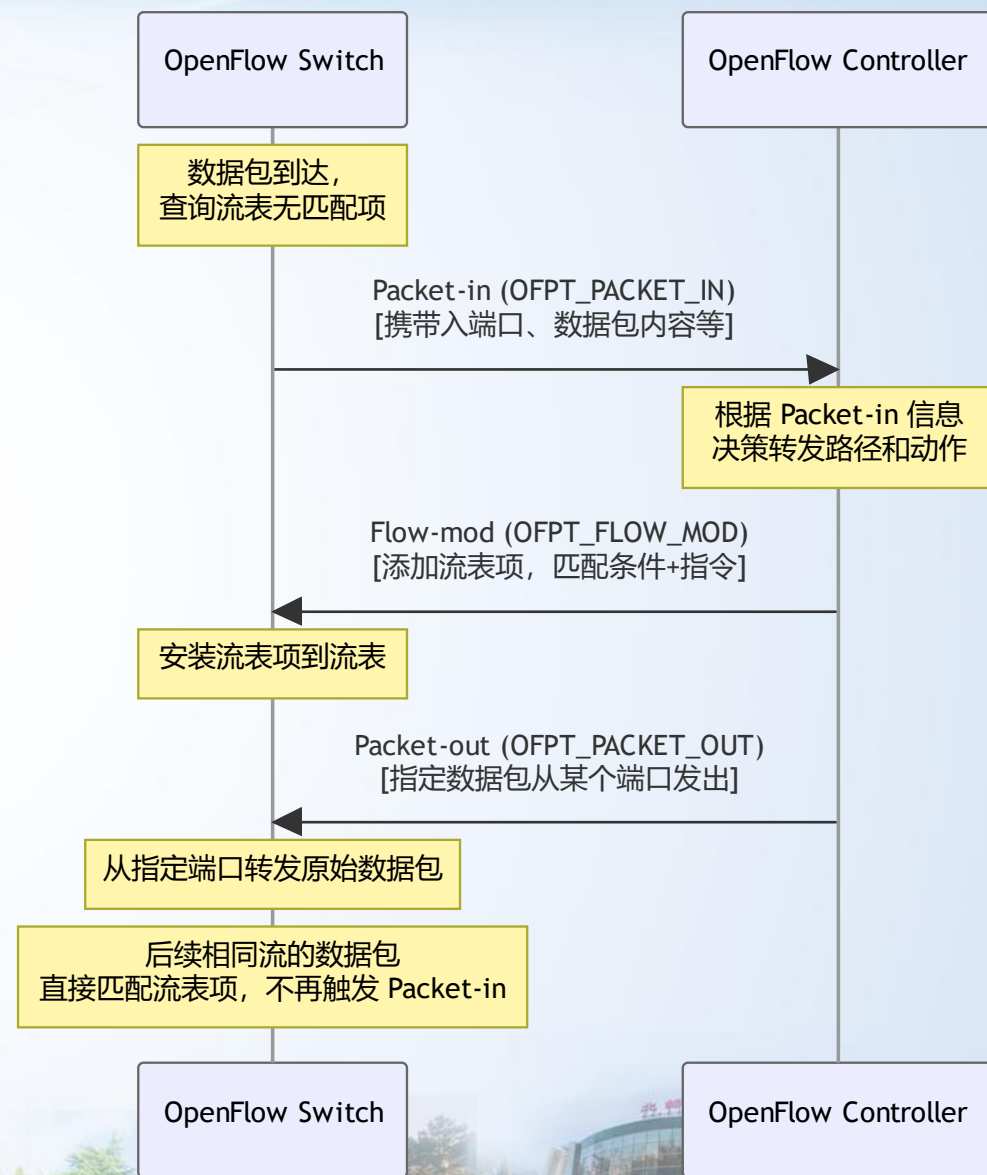
■ 交换机主动发起：

- 理论上交换机也可发送 Echo Request 来检测控制器的可达性，但实际中较少使用

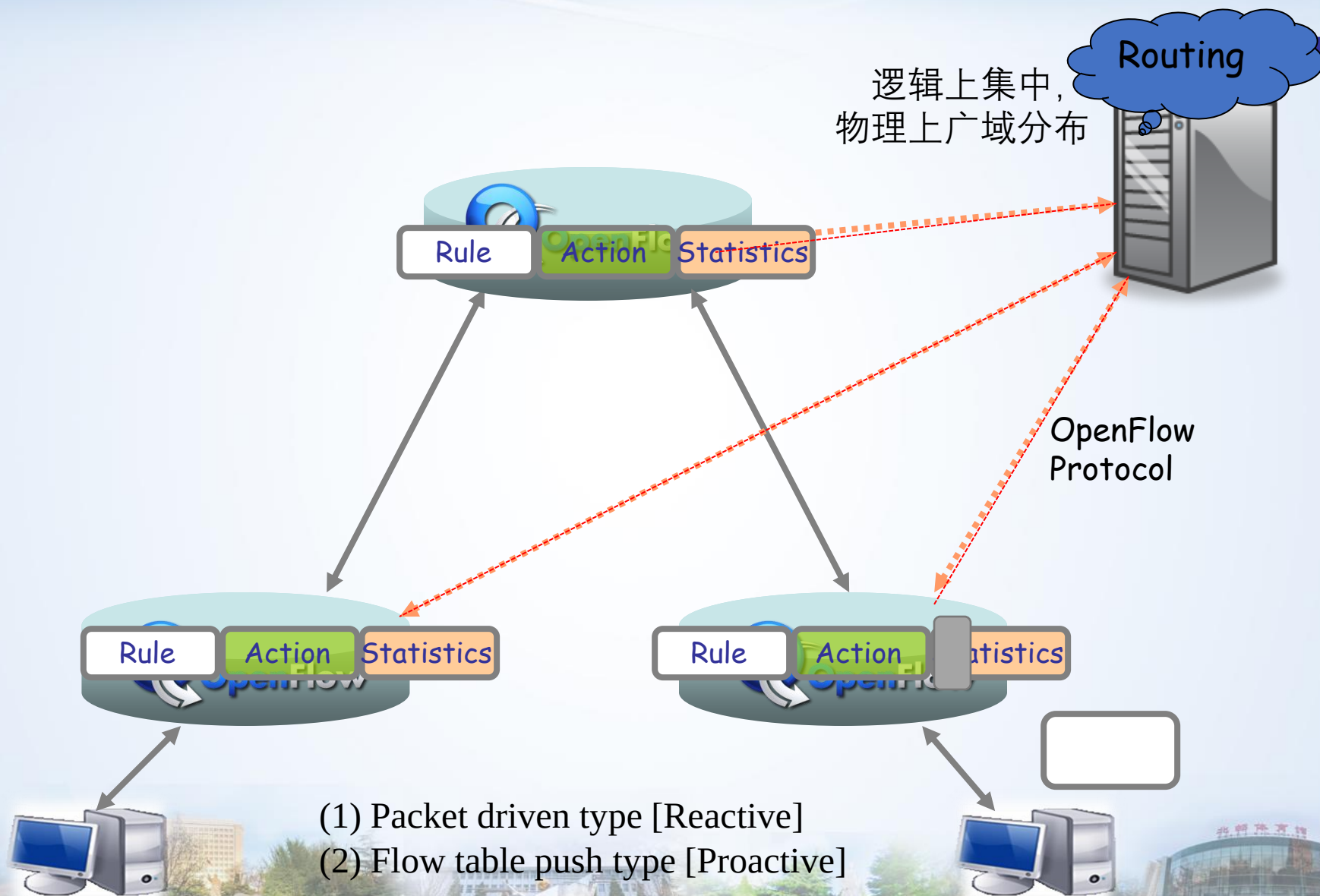


OpenFlow协议消息流程 —— 触发流表更新

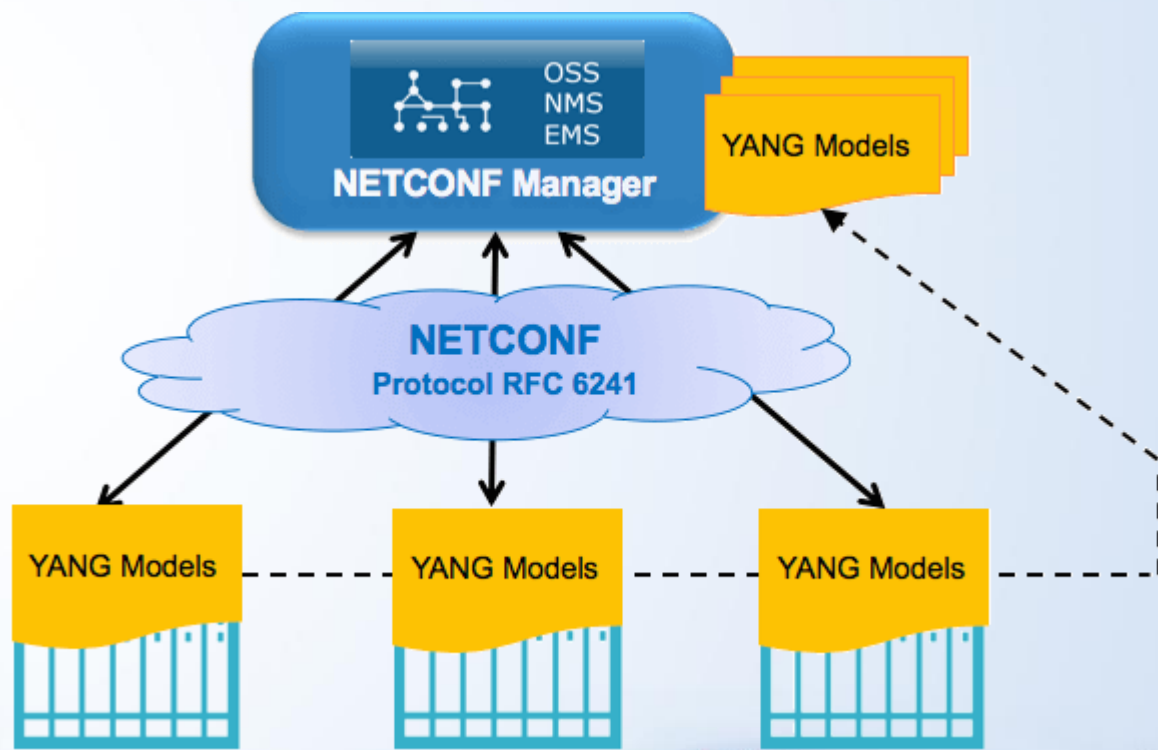
- 触发 **Packet-in**: 当交换机收到数据包且流表中没有匹配项时, 将数据包 (或部分内容) 封装在 Packet-in 消息中发送给控制器
- 控制器决策: 控制器根据网络全局拓扑、策略等信息, 决定数据包的转发路径和如何安装流表
- 下发流表: 控制器通过 **Flow-mod** 消息向交换机下发一条或多条流表项, 用于指导后续同一条流的处理
- 转发原始数据包: 控制器通过 **Packet-out** 消息指示交换机将触发 Packet-in 的原始数据包从特定端口发出, 避免丢包
- 后续转发: 之后属于同一条流的报文到达交换机时, 直接命中已安装的流表项, 按照流表指令转发, 不再触发 Packet-in, 实现高速转发



Openflow 转发原理示意

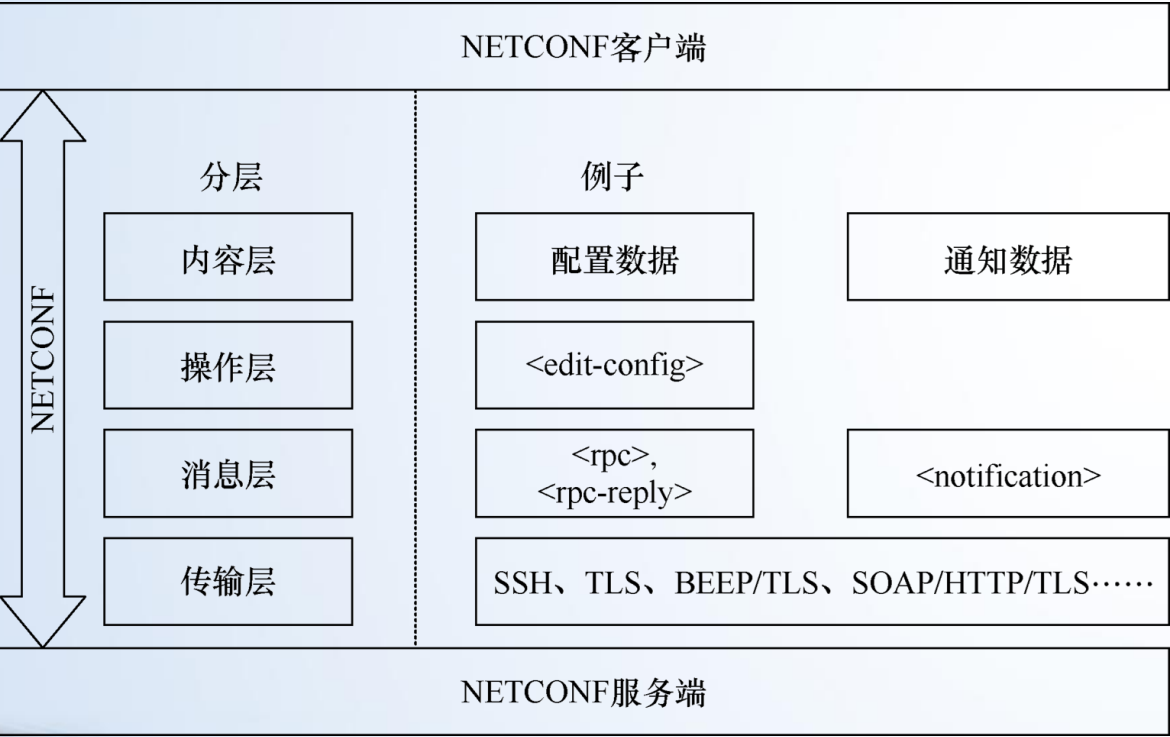


- NETCONF (Network Configuration Protocol, 网络配置协议) 在SDN中主要用于对网络设备进行**配置管理和状态监控**。它由IETF标准化, 采用**XML (可扩展标记语言)** 编码配置数据和**RPC (远程过程调用)** 机制, 与**YANG (下一代网络数据建模语言)** 数据模型紧密结合, 成为实现网络自动化和集中管理的关键技术之一
- NETCONF允许SDN控制器以编程方式统一配置网络设备, 替代传统命令行 (CLI) 手动登录的分散模式
- NETCONF还能实时获取设备的运行状态和统计信息
- NETCONF通常与YANG数据建模语言配合使用, 形成“YANG模型 + NETCONF协议”的标准化管理体系
- 在SDN的多租户环境中, NETCONF可用于动态创建和配置虚拟网络资源 (如VXLAN、VLAN、QoS策略), 为不同租户隔离配置空间



NETCONF协议的分层协议模型

■ NETCONF协议采用了**分层协议模型**，初始的配置数据通过每一层对应的协议进行包装



层面	说明
第一层：传输层	面向连接；身份认证、完整性和机密性；强制性运输协议。
第二层：消息层	NETCONF协议采用基于RPC的通信模式。NETCONF协议使用<rpc>和<rpc-reply>元素来提供与传输协议相独立的NETCONF请求和响应，来完成对网络设备的配置管理工作。
第三层：操作层	NETCONF协议在操作层中提供了一系列低级别操作来管理设备配置和检索设备状态信息，而设备状态信息一般存放在数据库中，这些操作提供了检索、配置、复制和删除配置数据存储的功能。
第四层：内容层	内容层由配置数据和通知数据组成。在NETCONF的标准规定中并没有对内容层进行标准化，但是在RFC 6020标准中提出的YANG数据建模语言被应用于NETCONF数据模型和协议操作。

使用NETCONF协议的配置示例

- 为网络设备上的一个名为 GigabitEthernet0/1 的接口配置IPv4地址 192.168.1.1/24
- 采用IETF标准化的YANG模型 **ietf-interfaces** 来定义接口数据，并通过NETCONF协议的 **<edit-config>** 操作将配置下发到设备

消息层
操作层

```
<?xml version="1.0" encoding="UTF-8"?>  
<rpc message-id="101" xmlns="urn:ietf:params:xml:ns:netconf:base:1.0">
```

发起配置请求

内容层

```
<edit-config>  
  <target>  
    <running/>  
  </target>  
  <config>  
    <interfaces xmlns="urn:ietf:params:xml:ns:yang:ietf-interfaces">  
      <interface>  
        <name>GigabitEthernet0/1</name>  
        <description>Configured by SDN Controller via NETCONF</description>  
        <type xmlns:ianaift="urn:ietf:params:xml:ns:yang:iana-if-type">ianaift:ethernetCsmacd</type>  
        <enabled>true</enabled>  
        <ipv4 xmlns="urn:ietf:params:xml:ns:yang:ietf-ip">  
          <address>  
            <ip>192.168.1.1</ip>  
            <prefix-length>24</prefix-length>  
          </address>  
        </ipv4>  
      </interface>  
    </interfaces>  
  </config>  
</edit-config>  
</rpc>
```

基于YANG模型

```
<?xml version="1.0" encoding="UTF-8"?>  
<rpc-reply message-id="101" xmlns="urn:ietf:params:xml:ns:netconf:base:1.0">  
  <ok/>  
</rpc-reply>
```

配置成功响应

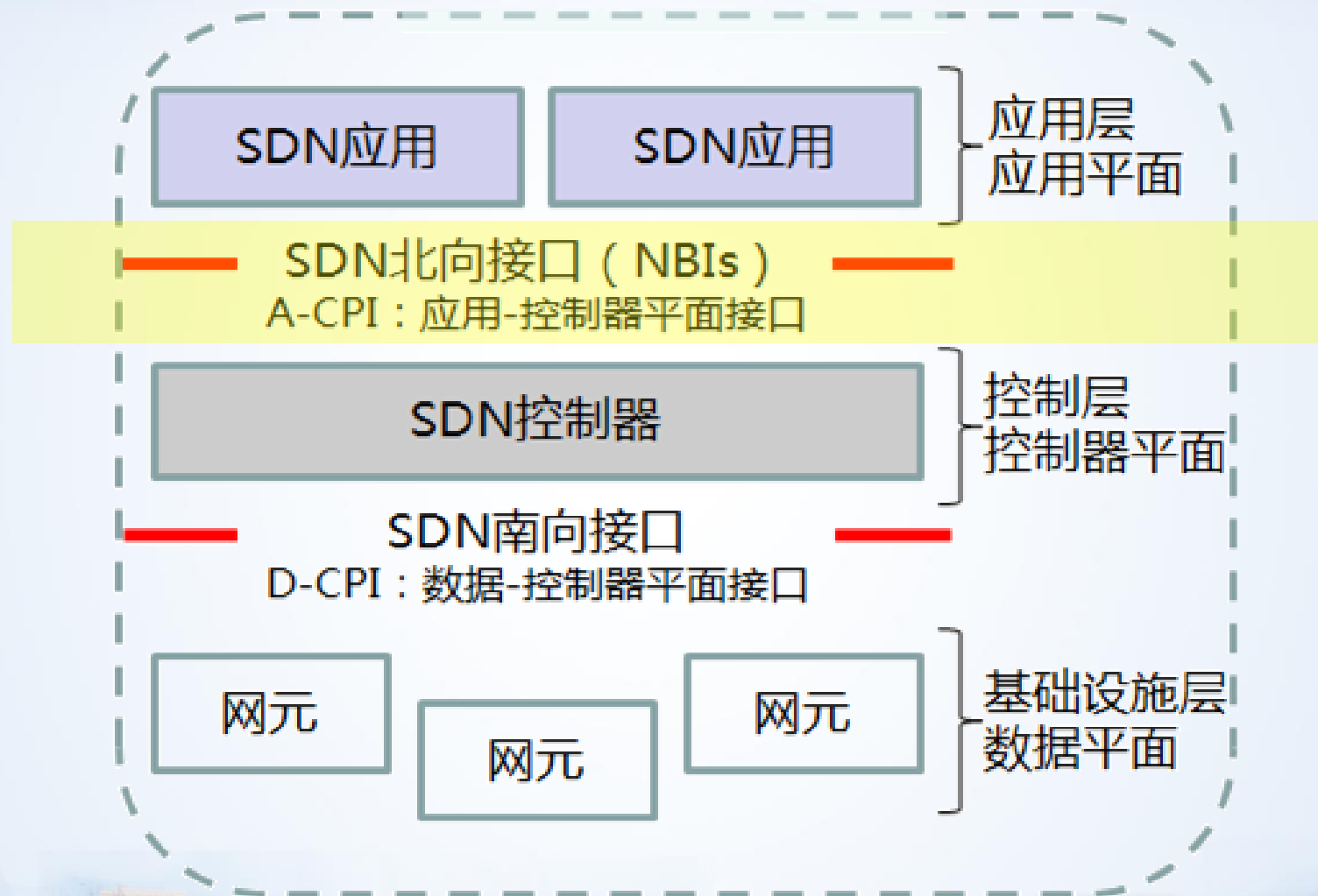
YANG模型作用和示例

- YANG是一种用于网络设备的数据建模语言，由IETF标准化（RFC 6020/RFC 7950）
- YANG定义了数据的结构和约束，但并不规定数据的传输方式
- YANG为不同厂商的网络设备提供了统一的数据描述语言。只要设备实现了相同的YANG模型（例如IETF标准的接口模型[ietf-interfaces](#)），控制器就可以用相同的方式配置这些设备，无需关心底层实现差异
- 右侧模型定义了一个名为interfaces的容器，其中包含一个接口列表，每个接口有名称、启用状态和IPv4地址（地址+掩码长度）。控制器可以通过NETCONF发送符合此模型的XML数据，完成对接口的配置

```
module example-interface {  
  yang-version 1.1;  
  namespace "http://example.com/ns/example-interface";  
  prefix "exif";  
  
  import ietf-inet-types {  
    prefix inet;  
  }  
  
  container interfaces {  
    list interface {  
      key "name";  
      leaf name {  
        type string;  
      }  
      leaf enabled {  
        type boolean;  
        default true;  
      }  
      container ipv4 {  
        leaf address {  
          type inet:ipv4-address;  
        }  
        leaf prefix-length {  
          type uint8 {  
            range "0..32";  
          }  
        }  
      }  
    }  
  }  
}
```

说明:

- **容器 (Container)**：将相关节点组织在一起，类似JSON或XML中的对象
- **列表 (List)**：定义多个同类型实例的集合，每个实例由唯一键标识（例如接口列表，每个接口有唯一的名字）
- **叶子 (Leaf)**：包含单一值的节点，如整数、字符串、IP地址等
- **叶子列表 (Leaf-list)**：包含多个值的节点序列



- 北向接口是SDN架构中连接应用层与控制层的桥梁，它为开发者提供了一组基于网络高层逻辑抽象与业务模型的编程接口。通过这套接口，开发者能够以标准化的方式与SDN控制器进行交互，实现对网络行为的灵活定义和动态调度
- 从标准化角度看，开放网络基金会（ONF）定义的北向接口致力于提供一种与厂商无关的抽象网络视图。它将底层网络资源封装为统一的服务模型，使上层应用无需关心具体设备的差异，从而实现业务意图的快速部署与跨平台互通
- 在具体实现层面，REST API是目前最为广泛的北向接口形态。它基于HTTP协议，以轻量、简洁的方式将控制器的网络能力开放为可调用的资源接口。开发者可以通过熟悉的GET/POST等操作，轻松实现对拓扑查询、流表下发、策略配置等功能的二次开发

- **REST API**也称为RESTful API，是遵循REST架构规范的**应用编程接口**(API或WebAPI)，支持与RESTful Web服务进行交互。
- **REST**是**表述性状态传递**的英文缩写，由计算机科学家Roy Fielding创建
- **REST**是一组**架构规范**，并非协议或标准，API开发人员可以采用各种方式实施REST
- **REST**使用了**HTTP或HTTPS**
 - 在 RESTful 设计中，HTTP 的各种方法被赋予了明确的语义，用于操作资源。**GET**：用于获取资源；**POST**：用于创建新资源；**PUT**和**PATCH**：用于更新资源（PUT 通常用于完整替换，PATCH 用于部分更新）；**DELETE**：用于删除资源。
 - HTTP 的状态码（如 **200 OK**、**404 Not Found**、**500 Internal Server Error**）也被用来清晰地表示操作的结果
- REST用在SDN北向接口的设计中，可将**控制器基本功能模块和各网元看作网络资源**，对其进行标识，通过增删查改的方法操作相应资源的数据
- **Ryu、OpenDaylight、Floodlight**等控制器都使用REST API作为北向接口

- Ryu通过**ofctl_rest.py**、**rest_topology.py**等模块提供REST API
 - **ofctl_rest.py**: 提供了一组 REST API, 用于查询和配置 OpenFlow 交换机的状态, 例如获取交换机列表、流表项、端口统计信息等
 - **rest_topology.py**: 提供了查询网络拓扑信息的 REST API, 例如交换机和链路的连接关系
 - **rest_firewall.py**: 实现了一个基于 REST API 的防火墙应用, 允许通过 HTTP 请求添加或删除防火墙规则
 - **rest_qos.py**: 提供了基于 REST API 的 QoS (服务质量) 管理功能
 - **rest_router.py**: 实现了一个基于 REST API 的三层路由器功能

- **ryu.app.ofctl_rest**为Ryu提供一套基于REST API的北向接口
- ofctl_rest提供三类REST API接口，基础信息查询类、流表操作类、高级功能类（组表、计量表）

基础信息查询类

API名称	请求方法/URL	参数	功能说明
获取所有交换机	GET /stats/switches	无	返回所有连接的交换机DPID列表
获取交换机描述	GET /stats/desc/<dpid>	dpid(交换机ID)	获取指定交换机的制造商、硬件描述等信息
获取流表统计	GET /stats/flow/<dpid>	dpid	获取指定交换机的所有流表项统计
获取端口统计	GET /stats/port/<dpid>	dpid	获取指定交换机的端口收发数据统计
获取组表统计	GET /stats/group/<dpid>	dpid	获取指定交换机的组表统计信息
获取计量表统计	GET /stats/meter/<dpid>	dpid	获取指定交换机的计量表统计信息
获取表统计	GET /stats/table/<dpid>	dpid	获取指定交换机的流表配置统计

- **ryu.app.ofctl_rest**为Ryu提供一套基于REST API的北向接口
- **ofctl_rest**提供三类REST API接口，基础信息查询类、流表操作类、高级功能类（组表、计量表）

流表操作类

API名称	请求方法/URL	参数（JSON格式）	功能说明
添加流表项	POST /stats/flowentry/add	dpid, match, actions, priority等	向指定交换机添加一条流表项
修改流表项	POST /stats/flowentry/modify	dpid, match, actions	修改所有匹配的流表项
删除匹配流表项	POST /stats/flowentry/delete	dpid, match	删除所有匹配指定条件的流表项
清空所有流表	DELETE /stats/flowentry/clear/<dpid>	dpid	删除指定交换机的所有流表项

高级功能类（组表、计量表）

API名称	请求方法/URL	参数	功能说明
添加组表项	POST /stats/groupentry/add	dpid, group_id, buckets等	添加组表项，用于多路径转发
添加计量表	POST /stats/meterentry/add	dpid, meter_id, bands	添加计量表项，用于QoS限速

- **rest_topology.py**提供了查询网络拓扑信息的REST API

接口名称 (URL)	HTTP 方法	功能描述
/v1.0/topology/switches	GET	获取当前网络中所有交换机的列表
/v1.0/topology/switches/{dpid}	GET	获取指定 DPID 的交换机详细信息
/v1.0/topology/links	GET	获取网络中所有的链路 (Link) 信息
/v1.0/topology/links/{dpid}	GET	获取与指定 DPID 交换机相关联的链路信息
/v1.0/topology/hosts	GET	获取网络中发现的所有主机 (Host) 信息
/v1.0/topology/hosts/{dpid}	GET	获取连接到指定 DPID 交换机上的主机信息

Ryu的REST API使用示例 —— 添加流表项

- 添加流表项: **POST /stats/flowentry/add**

/stats/flowentry/add接口参数

Attribute	Description	Example	Default
dpid	Datapath ID (int)	1	(Mandatory)
cookie	Opaque controller-issued identifier (int)	1	0
cookie_mask	Mask used to restrict the cookie bits (int)	1	0
table_id	Table ID to put the flow in (int)	0	0
idle_timeout	Idle time before discarding (seconds) (int)	30	0
hard_timeout	Max time before discarding (seconds) (int)	30	0
priority	Priority level of flow entry (int)	11111	0
buffer_id	Buffered packet to apply to, or OFP_NO_BUFFER (int)	1	OFP_NO_BUFFER
flags	Bitmap of OFPFF_* flags (int)	1	0
match	Fields to match (dict)	{"in_port":1}	{ } #wildcarded
actions	Instruction set (list of dict)	[{"type":"OUTPUT", "port":2}]	[] #DROP

- 添加流表项: **POST /stats/flowentry/add**

使用示例:

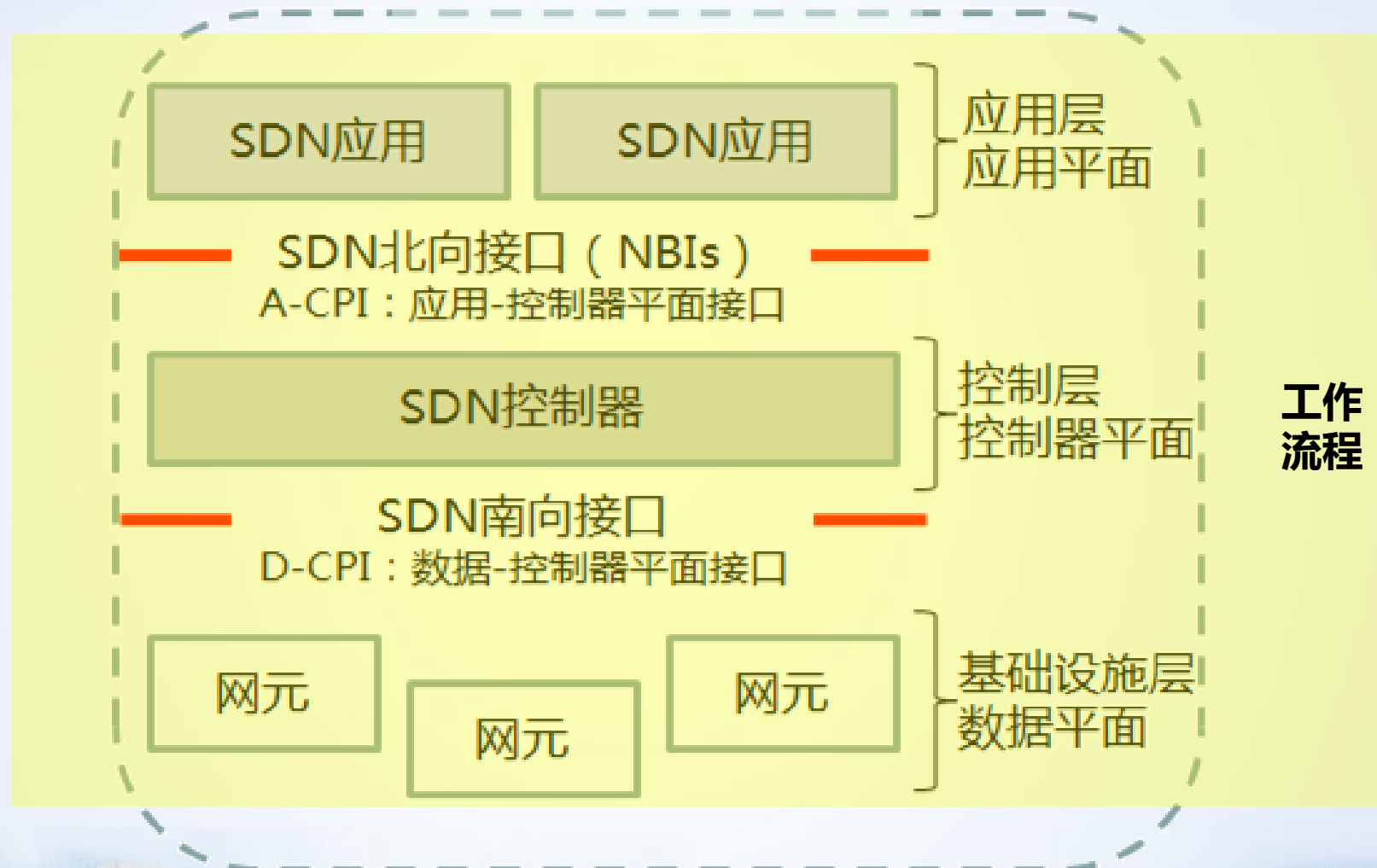
```
$ curl -X POST -d '{
    "dpid": 1,
    "cookie": 1,
    "cookie_mask": 1,
    "table_id": 0,
    "idle_timeout": 30,
    "hard_timeout": 30,
    "priority": 11111,
    "flags": 1,
    "match":{
        "in_port":1
    },
    "actions":[
        {
            "type":"OUTPUT",
            "port": 2
        }
    ]
}' http://localhost:8080/stats/flowentry/add
```


- RESTCONF本质上是NETCONF协议基于HTTP的RESTful版本。两者都使用YANG模型定义数据，但RESTCONF在编码上同时支持XML和JSON，因此更轻量、易读
- Ryu本身不支持RESTCONF，需自行扩展
- **RESTCONF 通常作为 北向接口**
- **NETCONF 通常作为 南向接口**

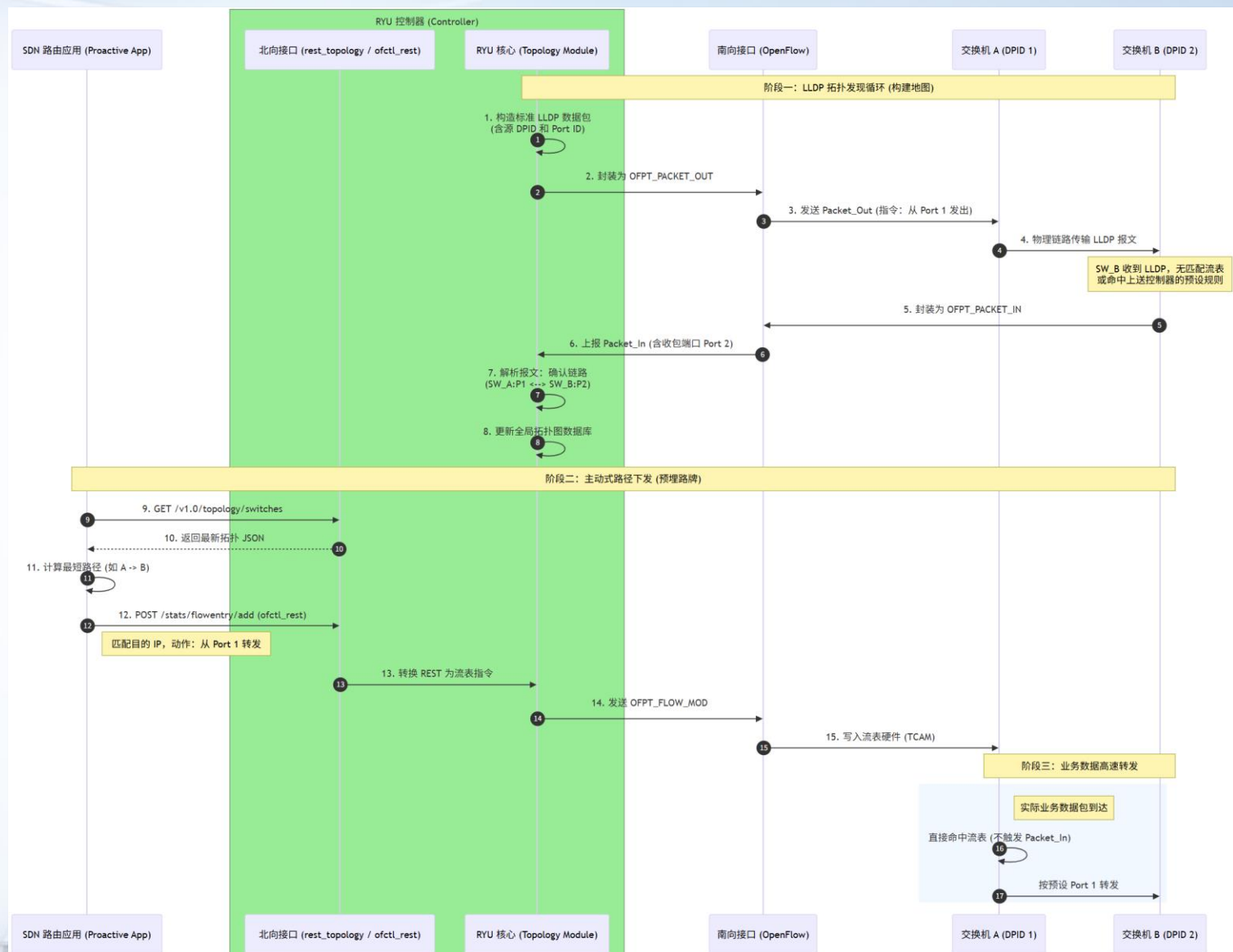
```
PATCH /restconf/data/ietf-  
interfaces:interfaces/interface=GigabitEthernet0/1 HTTP/1.1  
Host: <device-ip>  
Content-Type: application/json  
Accept: application/json
```

```
{  
  "ietf-interfaces:interface": {  
    "name": "GigabitEthernet0/1",  
    "description": "Configured by SDN Controller via  
NETCONF",  
    "type": "iana-if-type:ethernetCsmacd",  
    "enabled": true,  
    "ietf-ip:ipv4": {  
      "address": [  
        {  
          "ip": "192.168.1.1",  
          "prefix-length": 24  
        }  
      ]  
    }  
  }  
}
```

基于YANG模型

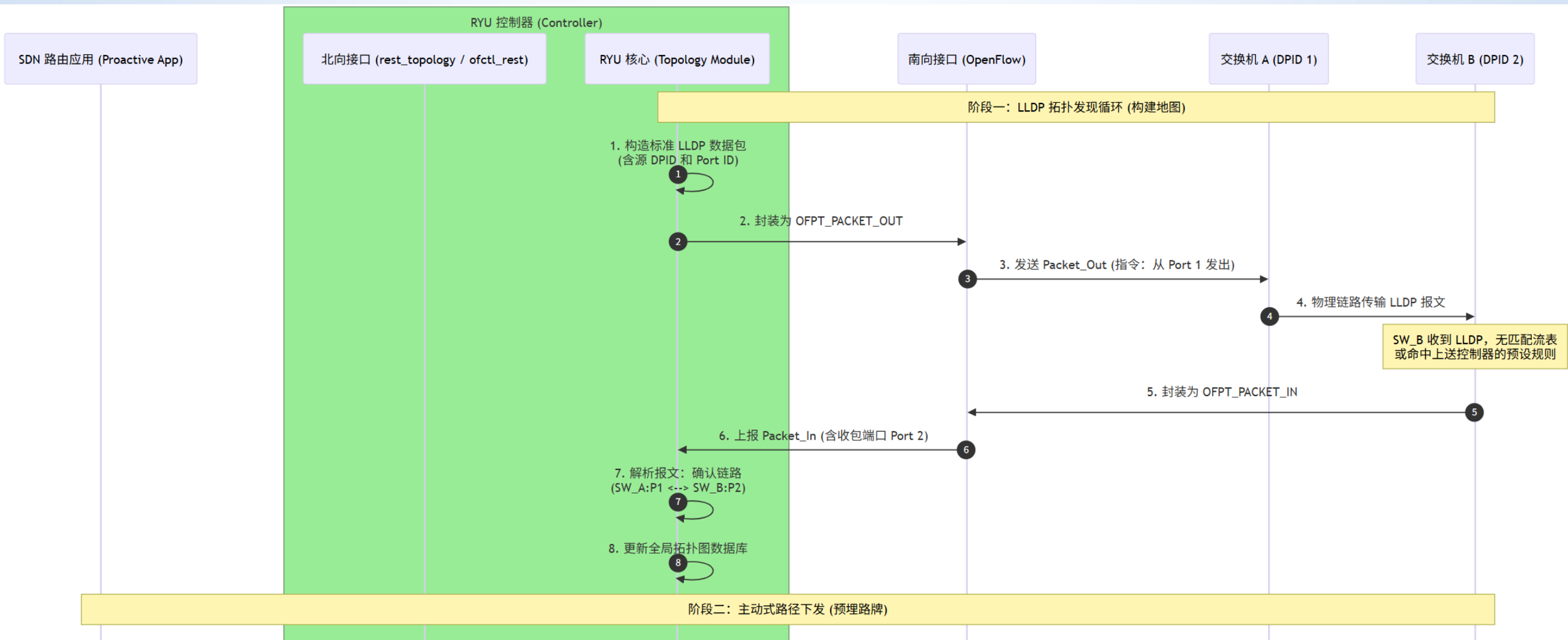


- 三层架构（业务平面、控制平面、转发平面）+ 两个接口（南向、北向）的工作流程
- 以Ryu控制器的北向接口为例，构建“**最短路径转发**”应用，整体流程包括三个阶段
 - LLDP拓扑发现
 - 主动路径下发
 - 业务数据高速转发



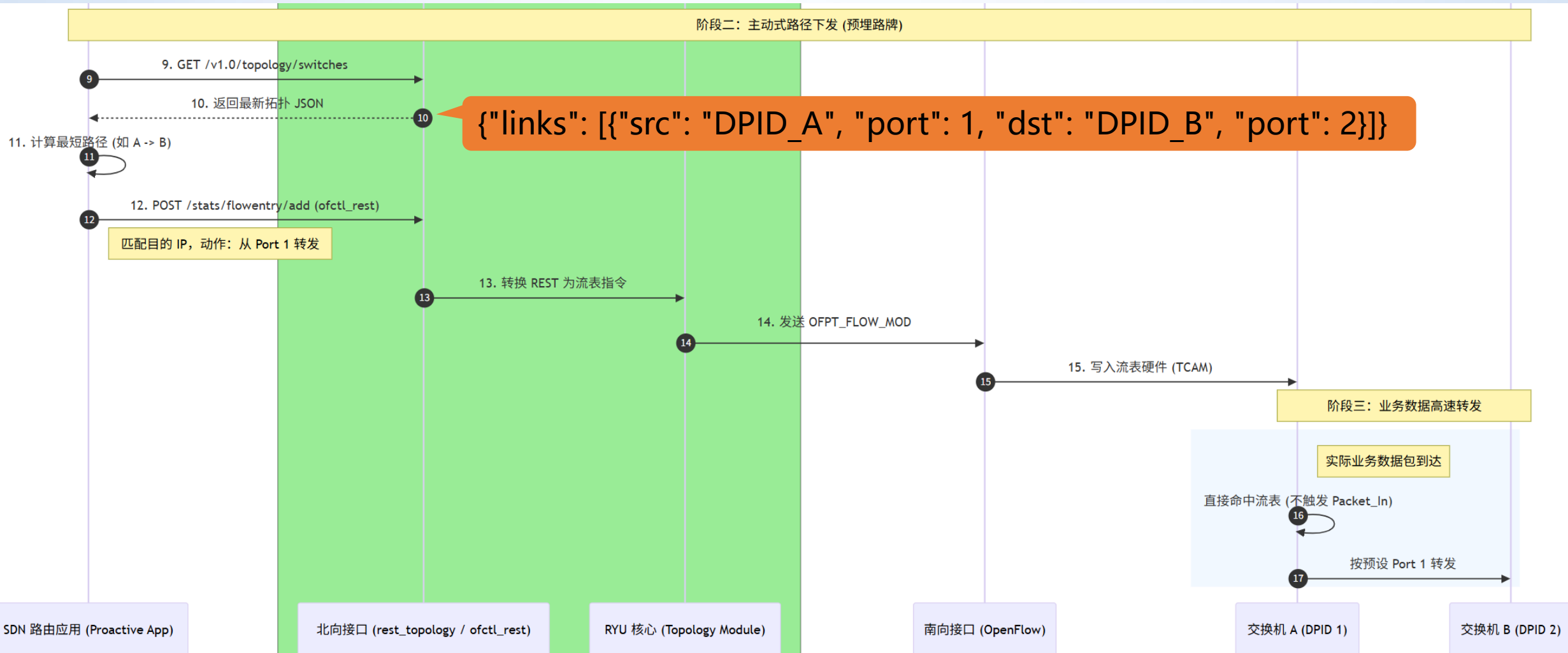
SDN工作流程 —— LLDP拓扑发现

● 阶段一：SDN控制器使用构建LLDP进行拓扑发现



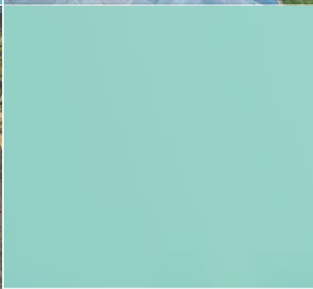
SDN工作流程 —— 路径下发 + 业务数据高速转发

- 阶段二：基于北向接口进行路径下发，更新流表
- 阶段三：基于流表进行业务数据的高速转发



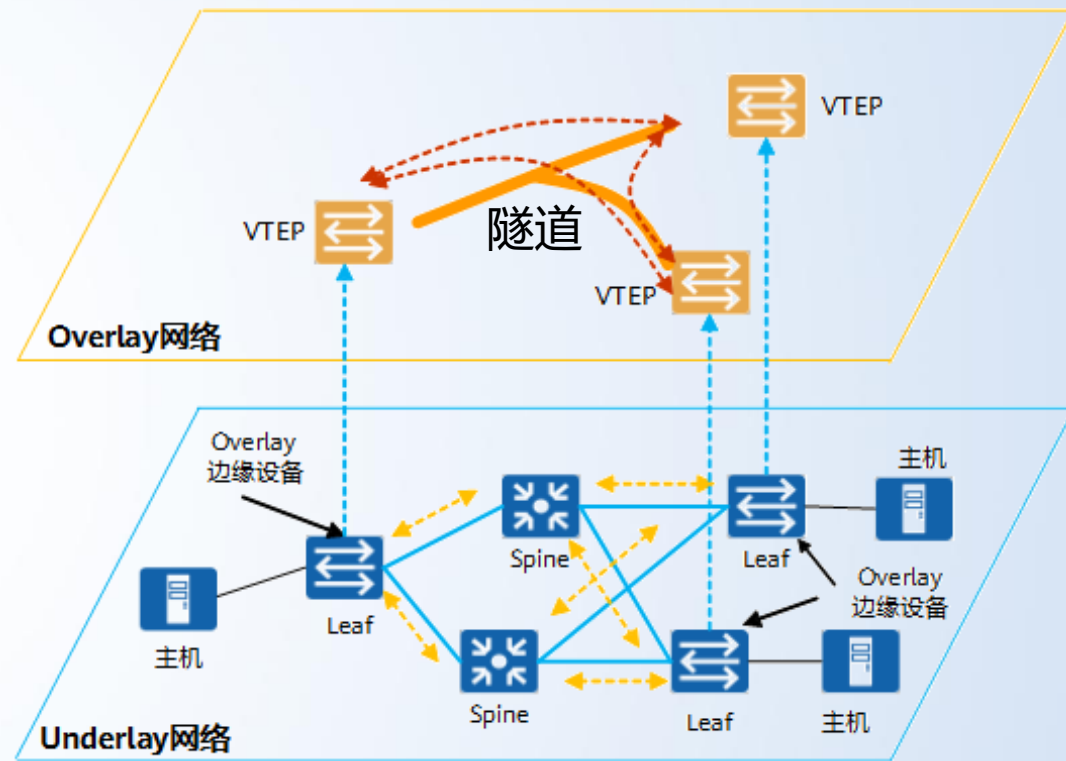
SDN的应用

04



SDN在数据中心网络的应用 — Overlay网络

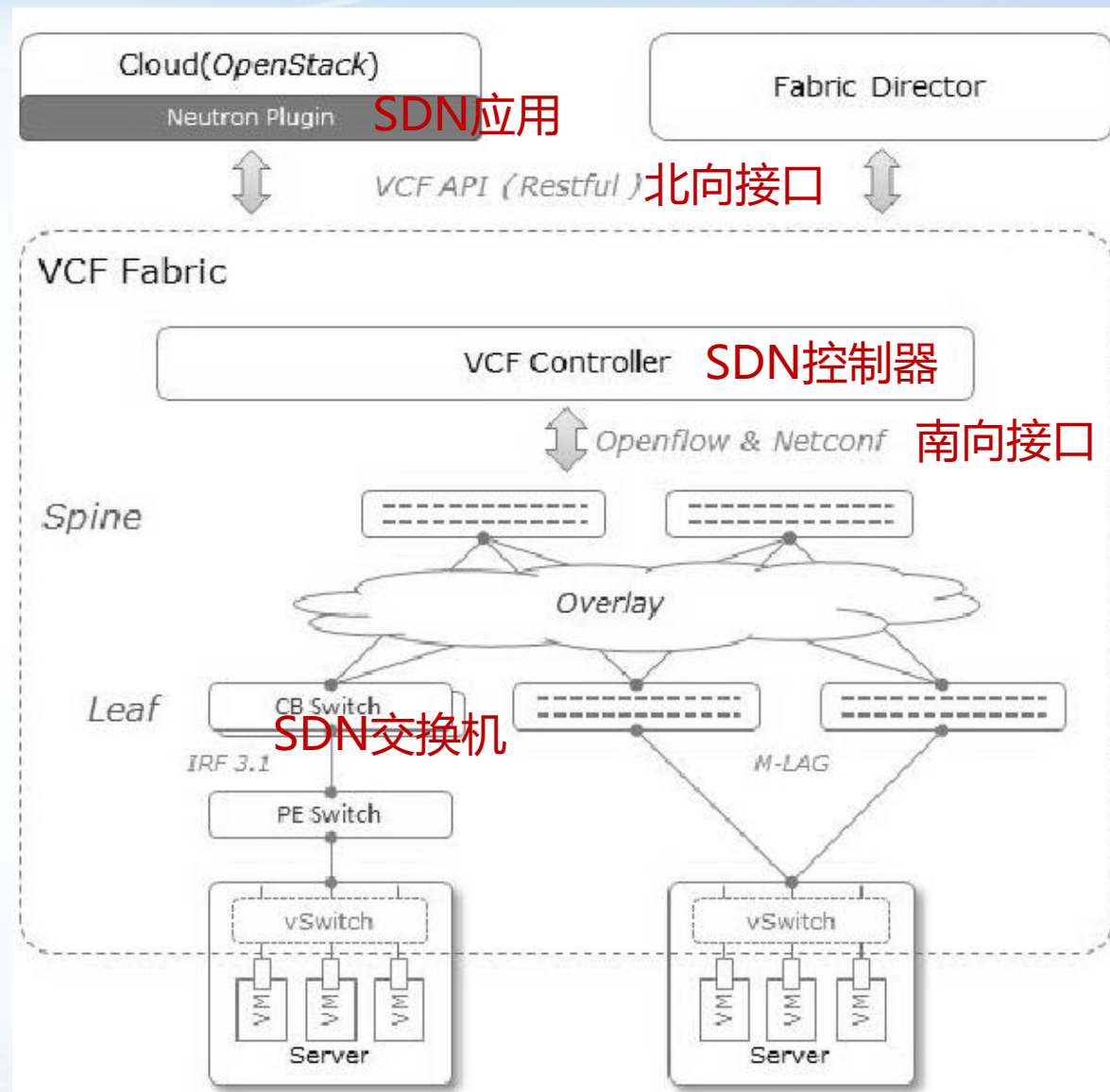
- **Underlay**网络可以由多个类型设备互联而成的物理网络，负责网络之间的数据包传输
- **Overlay**网络是通过网络虚拟化技术，在同一张Underlay网络上构建出的一张或者多张虚拟的逻辑网络
- 相互连接的Overlay设备之间**建立隧道**，数据包准备传输出去时，设备为数据包添加新的IP头部和隧道头部，并且被屏蔽掉内层的IP头部，数据包根据新的IP头部进行转发
- 当数据包传递到另一个设备后，外部的IP报头和隧道头将被丢弃，得到原始的数据包，在这个过程中Overlay网络并不感知Underlay网络
- Overlay网络有着各种网络协议和标准，包括**VXLAN**、NVGRE、SST、GRE、NVO3、EVPN等，如VXLAN 能把“二层网络”的数据包，封装在“三层 IP 网络”里传输，从而跨越物理网络限制，创建出大量相互隔离的虚拟网络



Leaf节点作为Overlay网络的边缘设备承担VTEP (VXLAN Tunnel EndPoint) 的角色

SDN在数据中心网络的应用 — SDN赋能Overlay网络

- 在传统网络中，维护隧道协议（如 VXLAN）的映射表非常痛苦。但在 SDN 环境中，**控制器可以直接向 Leaf 节点（即 VTEP）下发流表**，自动完成隧道路径的计算与建立，实现了 **Overlay 网络的自动化编排**
- SDN 控制器负责“逻辑计算”，将网络策略转化为流表；Leaf 节点执行“物理封装”，利用 VTEP 功能将业务流量送入隧道
- Overlay 屏蔽了物理网络的异构性，而 SDN 则屏蔽了隧道管理的复杂性



SDN在广域网的应用 — SD-WAN

SD-WAN

软件定义广域网 (Software-Defined WAN)

传统 WAN 的痛点分析

成本昂贵: MPLS 带宽单价高, 扩容周期长

架构僵化: 流量回传总部导致云访问高延迟

配置复杂: 逐台手工配置, 运维难度大

缺乏灵活性: 难以根据应用需求动态分配带宽

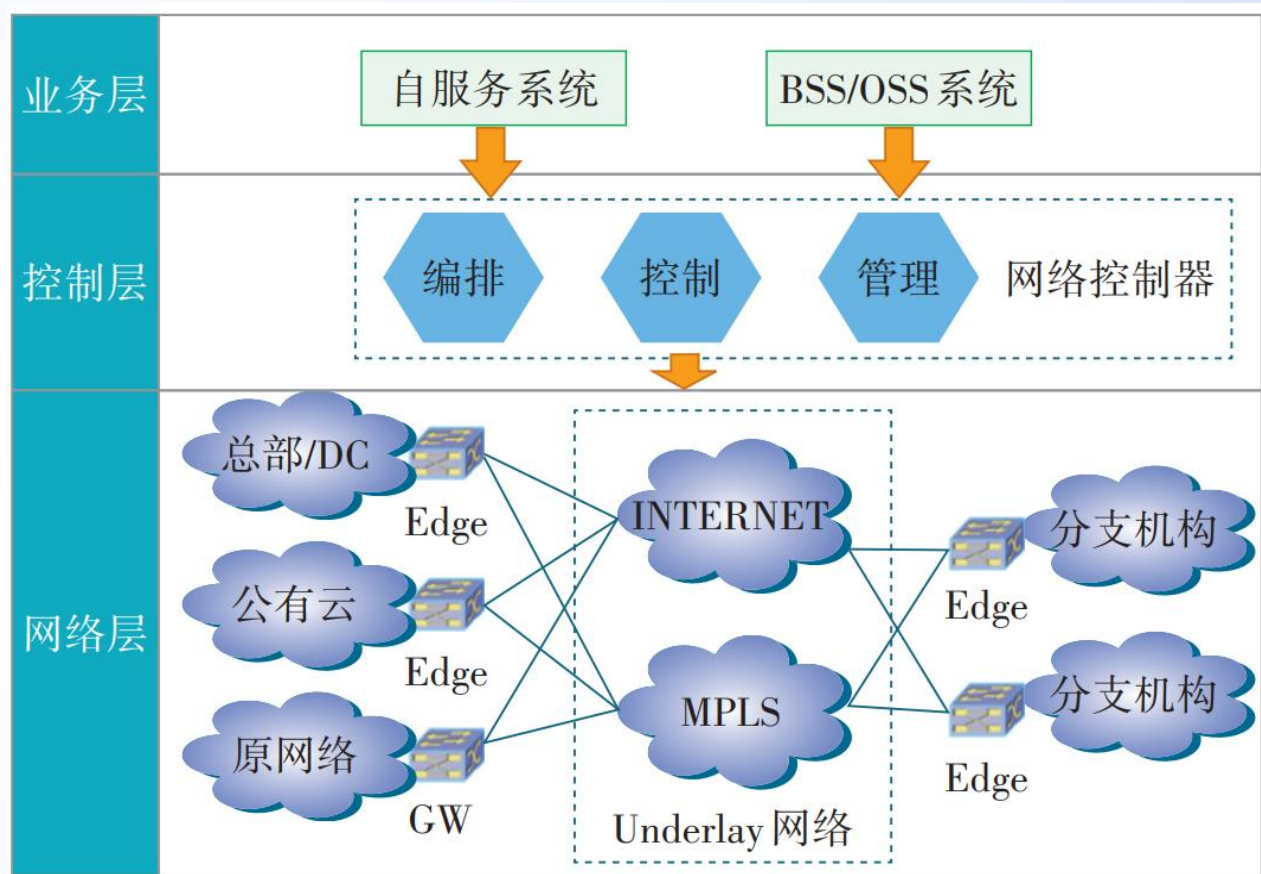
SDN 在 SD-WAN 中的关键作用

转控分离: 解耦数据平层与控制平层, 将路径决策权收归控制器, 优化转发效率

集中管理: 全网策略统一下发, 替代单机逐一配置模式, 实现业务分钟级上线

可编程性: 软件定义路径, 支持基于丢包率、时延等链路质量指标实现动态选路

零接触部署 (ZTP): 设备即插即用, 自动化获取配置与软件版本, 大幅降低异地运维难度



SDN在广域网的应用 — SD-WAN

业务层

企业管理员可以直接定义业务意图

控制层

控制层是“软件定义”的核心，它的主要组件是网络控制器，它一般具有业务编排、网络控制和网络管理功能

控制器将业务层的“意图”翻译成底层的“流表”或“配置”，实现对广域网流量的精准操控

网络层

Underlay 网络：混合了 INTERNET 和 MPLS 专线

Edge/GW 节点：部署在总部、公有云或分支机构的“边缘设备”，是 **SDN 的转发单元**。它们根据控制器的指令，动态地在不同的 Underlay 链路上建立隧道（使用SRv6、VXLAN等），实现流量的智能调度

